2026

UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308: Data Structures and Algorithms I & II

TECHNICAL REPORT

GHANA SMART SERVICE OPERATIONS OPTIMIZER

Sub-Title: Legon Campus Smart Service & Dispatch Operations Hub

STUDENT ID	FULL NAME
22098316	Lorretta Opoku Nsiah
22412662	Van-Kpikpi, Vanessa Selasi
22056801	Michael Gorswin Achel
22128981	Gideon Elorm Glago
22233989	Kwabena Awuah Bosompem
22069268	Asantewaa Isabella
22299460	Asare Bismark
22309657	Duah, Ebenezer Ohene Amoako
22304655	Asante, Stephanhy
22396017	Amenumey, Jude Kwame Enam
22406063	Arhinful, Solomon Kwesi
22375144	Kumah, Michael Nhyira

22382281	Quayson, Isaac Awortwe
22388561	Akyea, Benjamin Obeng

1. Executive Summary & Local Context

The Legon Campus Smart Service Operations Optimizer is an integrated operational platform tailored for the University of Ghana, Legon campus. Campus service teams — the Physical Development and Municipal Services Directorate (PDMSD), the University of Ghana Computing Systems (UGCS), maintenance teams, security patrols, and campus shuttle logistics — routinely face complex coordination problems. Service requests arrive from academic departments, lecture halls, science laboratories, student hostels, and municipal junctions with varied urgency and strict deadlines.

This system replaces ad-hoc operations with algorithmic decision-making, through five core capabilities:

- Dynamic Service Dispatching — prioritizing urgent repairs and tasks using custom FIFO queues, circular buffers, deques, and min-heaps.
- Optimal Campus Routing — evaluating the quickest paths across 58 campus locations and 117 interconnected road segments using Dijkstra's algorithm, with travel-time and road-condition penalties.
- Network Infrastructure Optimization — generating campus-wide minimum-cost distribution and communication backbones using Kruskal's and Prim's Minimum Spanning Tree algorithms, backed by custom Disjoint Sets with path compression.
- Constrained Resource Allocation — maximizing resolved emergency request values under shift-hour and crew-capacity constraints, via 0/1 Knapsack Dynamic Programming.
- Zero Java-Collection Core — all 13 fundamental linear, tree, priority, and graph data structures were implemented from first principles in Java 21, coupled with SQLite persistent storage and an empirical efficiency benchmarking lab.

2. Problem Statement, Assumptions & Input–Output Specifications

2.1 Core Operational Questions

- Q1 (Priority & Scheduling): Given an arriving stream of campus maintenance requests, which job must be dispatched next to minimize penalty fines and prioritize critical laboratory emergencies?
- Q2 (Shortest Path Routing): What is the fastest traversal route from the Central Administration / Great Hall to distant residential zones (e.g., Jean Nelson, Pentagon, or UGMC), factoring in road condition weights?
- Q3 (Zone Reachability): Which campus facilities are reachable within k road hops from a municipal dispatch center during road blockages or maintenance?
- Q4 (Budget/Capacity Optimization): Given a 240-minute maintenance crew shift and 157 pending requests, what exact subset of jobs maximizes total avoided penalty fines?
- Q5 (Empirical Scalability): How do custom data structure operations scale from N = 100 to N = 10,000+ items compared to theoretical asymptotic bounds?

2.2 System Boundaries & Assumptions

- Node Network: The Legon road network is represented as an undirected weighted graph G = (V, E) with |V| = 58 named campus locations and |E| = 117 validated road segments.
- Road Weights: Every road has a base travel time in minutes and a road condition weight factor w, ranging from 1.0 to 1.8. The total edge weight is computed as:

  $$\text{Cost} = \text{travelTime_min} \times \text{roadConditionWeight}$$

  All weights are strictly positive (greater than 0), which satisfies the precondition required for Dijkstra's algorithm to work correctly.
- Crew Shift Horizon: Maintenance shifts are modeled at standard 4-hour (240-minute) intervals, with a 15minute fixed service overhead per job.
- Persistence Boundary: CSV seed files initialize the SQLite relational database (legon_hub.db), and all subsequent state changes, runs, and audit undo logs are read from and written to SQLite tables.

2.3 Formal Input–Output Problem Specifications (5 Core Operations)

Each core operation below is defined by its inputs, its preconditions (what must be true before it runs), its outputs, and its postconditions (what is guaranteed to be true after it runs).

Operation 1 — ShortestPathDijkstra(G, sourceId, targetId)

Inputs

- G: custom Graph (adjacency list, V = 58, E = 117)
- sourceId: integer in [1, 58] (e.g., 1 for Great Hall)
- targetId: integer in [1, 58] (e.g., 58 for Engineering Annex)

Preconditions

- sourceId and targetId exist in G.locations
- all edge weights are greater than 0

Outputs

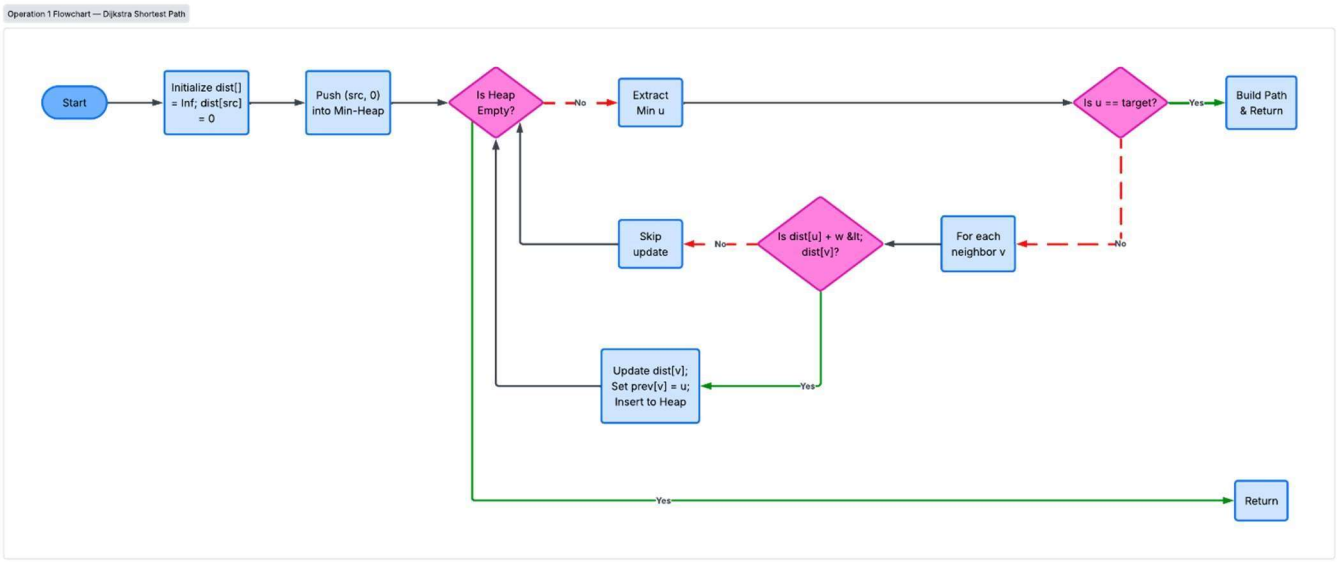
- path: dynamic array of location IDs representing the sequential route
- totalCost: floating-point value representing total time in minutes
- distanceTable: array of settled shortest costs from source to all V

Postconditions

- totalCost equals the minimum, over all valid paths, of the sum of edge weights along that path ● G remains unmodified

```
Algorithm: ShortestPathDijkstra(G, sourceId, targetId)
-------------------------------------------------------------------------------1.  Initialize dist[1..V] = +INFINITY, prev[1..V] =
NULL, visited[1..V] = FALSE
```

```
2.          dist[sourceId] = 0.0
3.          frontier = new MinHeap()
4.          frontier.insert(new Node(sourceId, 0.0))
5.          WHILE NOT frontier.isEmpty() DO:
6.          curr = frontier.extractMin()
7.          u = curr.locationId
8.          IF visited[u] == TRUE THEN CONTINUE
9.          visited[u] = TRUE
10.         IF u == targetId THEN BREAK  // Early exit optimization
11.         FOR EACH edge (u, v, weight) in G.adjList[u] DO:
12.         IF NOT visited[v] AND (dist[u] + weight < dist[v]) THEN:
13.         dist[v] = dist[u] + weight
14.         prev[v] = u
15.         frontier.insert(new Node(v, dist[v]))
16.         path = new DynamicArray()
17.         currNode = targetId
18.         WHILE currNode != NULL DO:
19.         path.addFirst(currNode)
20.         currNode = prev[currNode]
21.         RETURN (path, dist[targetId])
```



Operation 2 — ScheduleUrgentDispatch(requestStream, mode)

Inputs

- requestStream: custom dynamic array of ServiceRequest objects
- mode: one of FIFO, URGENCY_PRIORITY, DEADLINE_EARLIEST

Preconditions

- requestStream is non-null

Outputs

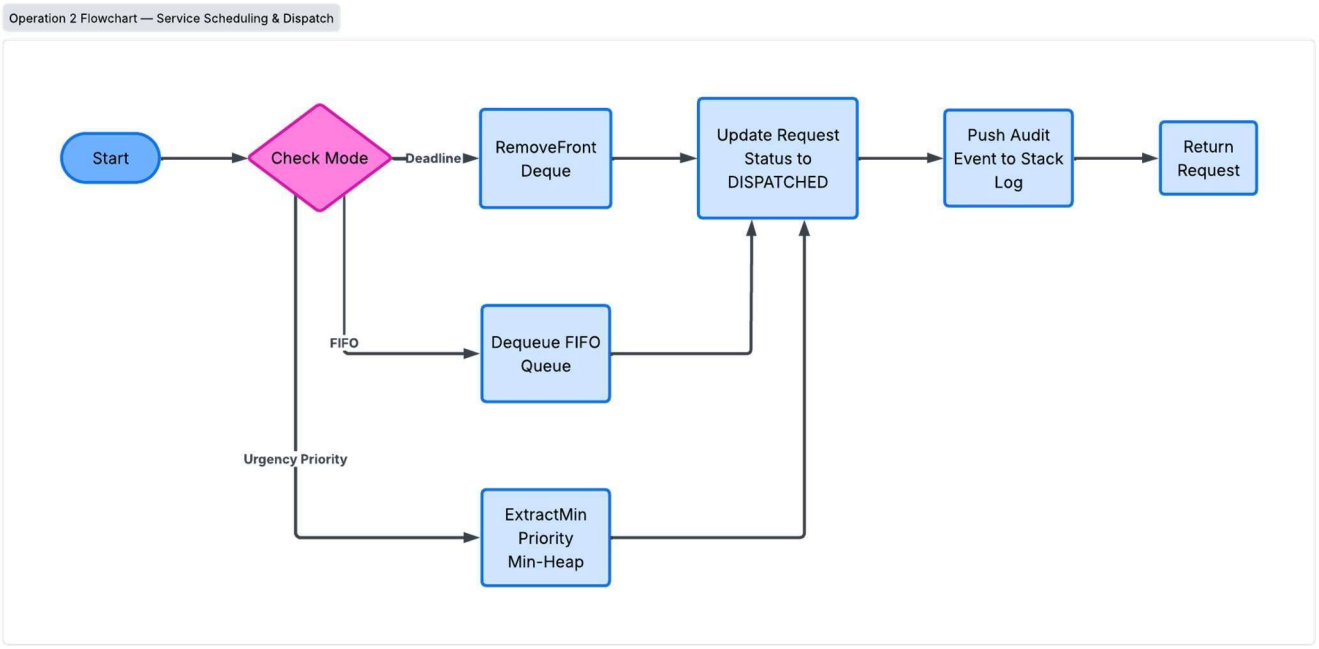
- dispatchedRequest: the ServiceRequest with the highest priority under the active policy

Postconditions

- under URGENCY_PRIORITY mode, the extracted item has the minimum score in the Min-Heap ● queue/heap size decreases by 1, and an AuditEvent is logged

```
Algorithm: ScheduleUrgentDispatch(requestStream, mode)
-------------------------------------------------------------------------------1.  IF mode == FIFO THEN:
2.          IF fifoQueue.isEmpty() THEN RETURN NULL
3.          dispatchedRequest = fifoQueue.dequeue()
4.          ELSE IF mode == URGENCY_PRIORITY THEN:
5.          IF priorityHeap.isEmpty() THEN RETURN NULL
6.          dispatchedRequest = priorityHeap.extractMin() // Min-Heap sorted by urgency score
7.          ELSE IF mode == DEADLINE_EARLIEST THEN:
8.          IF deadlineDeque.isEmpty() THEN RETURN NULL
```

```
9.          dispatchedRequest = deadlineDeque.removeFront()
10.         dispatchedRequest.status = "DISPATCHED"
11.         auditLogger.logEvent("DISPATCH", "Dispatched Request ID: " + dispatchedRequest.id)  12.
                RETURN dispatchedRequest
```



Operation 3 — MinimumSpanningNetwork(G, algorithmChoice)

Inputs

- G: custom Graph
- algorithmChoice: KRUSKAL or PRIM

Preconditions

- G is a connected, undirected graph with V vertices and E edges

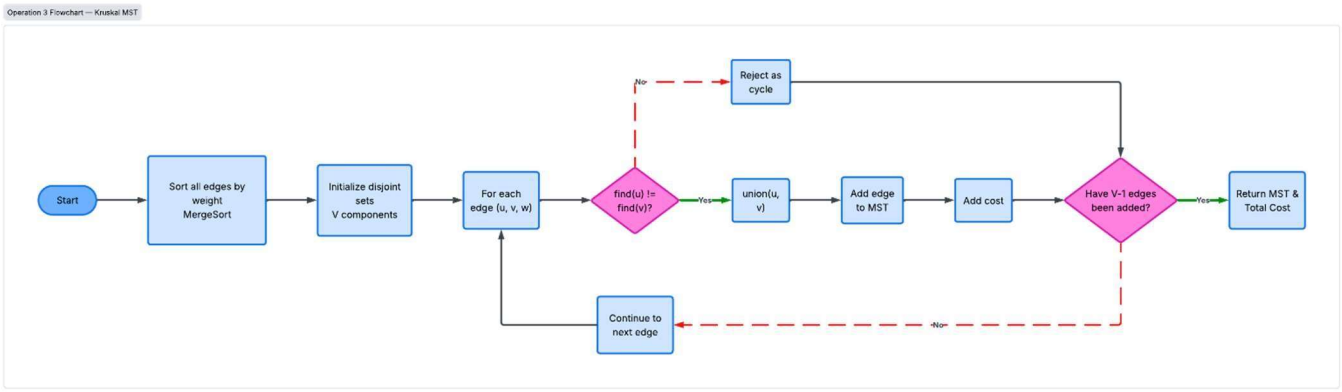
Outputs

- mstEdges: dynamic array of Road objects, containing exactly (V − 1) edges
- totalWeight: sum of weights of all selected MST roads

Postconditions

- the graph formed by mstEdges is acyclic and connects all V vertices
- totalWeight is minimal across all possible spanning subgraphs

```
Algorithm: MinimumSpanningNetwork_Kruskal(G)
-----------------------------------------------------------------------------
1.          mstEdges = new DynamicArray()
2.          totalCost = 0.0
3.          allEdges = G.getAllEdges() // Array of 117 road edges
4.          MergeSort(allEdges)       // Sort edges in ascending weight order O(E log E)  5.
            dset = new DisjointSet(G.numVertices) // Initialize 58 independent sets
6.          FOR EACH edge (u, v, weight) IN allEdges DO:
7.          rootU = dset.find(u)   // Find with Path Compression
8.          rootV = dset.find(v)  9.     IF rootU != rootV THEN:
10.     dset.union(rootU, rootV) // Union by Rank
11.     mstEdges.add(edge)
12.     totalCost = totalCost + weight
13.     IF mstEdges.size() == (G.numVertices - 1) THEN BREAK
14.     RETURN (mstEdges, totalCost)
```



Operation 4 — KnapsackResourceOptimizer(requests, shiftDurationMin, depotId) Inputs

- requests: array of N pending requests (N ≥ 1)
- shiftDurationMin: integer capacity W (e.g., 240 minutes)
- depotId: source dispatch location ID

Preconditions

- all request weights (travel time + service time) are greater than 0; fine values are ≥ 0

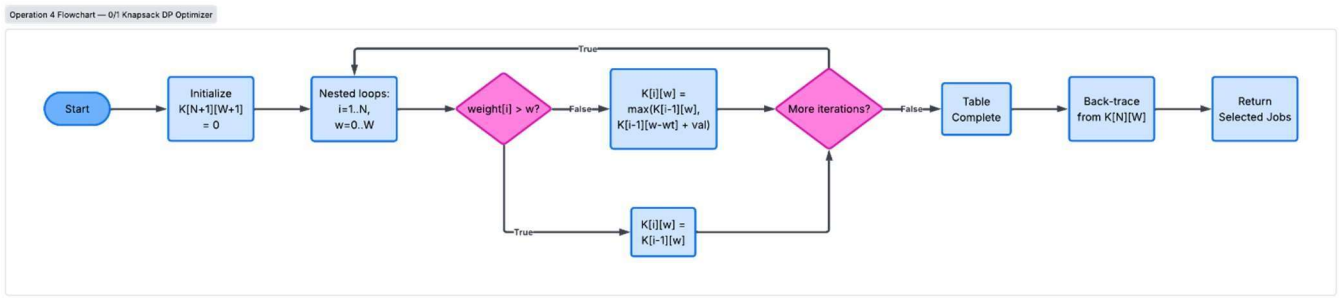
Outputs

- selectedJobs: dynamic array of chosen ServiceRequest objects
- totalValueEarned: sum of avoided fines and urgency bonuses
- totalTimeUsed: total duration ≤ shiftDurationMin

Postconditions

- the sum of selected job weights is ≤ W
- totalValue is mathematically optimal (guaranteed by the 0/1 Knapsack formulation)

```
Algorithm: KnapsackResourceOptimizer(requests, W)
-------------------------------------------------------------------------
1.      N = requests.length
2.      Initialize 2D array K[0..N][0..W] with 0
3.      FOR i = 1 TO N DO:
4.      wt = requests[i].travelTime + 15 // Travel time + service overhead
5.      val = requests[i].fineAmount + requests[i].urgencyBonus
6.      FOR w = 0 TO W DO:
7.      IF wt > w THEN:
8.      K[i][w] = K[i - 1][w]
9.      ELSE:
10.     K[i][w] = MAX(K[i - 1][w], K[i - 1][w - wt] + val)
11.     // Back-tracing to reconstruct optimal solution
12.     selectedRequests = new DynamicArray()
13.     currW = W
14.     FOR i = N DOWNTO 1 DO:
15.     IF K[i][currW] != K[i - 1][currW] THEN:
16.     selectedRequests.add(requests[i])
17.     currW = currW - (requests[i].travelTime + 15)
18.     RETURN (selectedRequests, K[N][W], W - currW)
-------------------------------------------------------------------------
```



Operation 5 — KeyRangeIndexSearch(BTree, minKey, maxKey)

Inputs

- BTree: custom B-Tree index structure (minimum degree t, derived from the team's index numbers)
- minKey, maxKey: search boundaries (e.g., request IDs or timestamps)

Preconditions

- minKey ≤ maxKey; B-Tree invariants hold

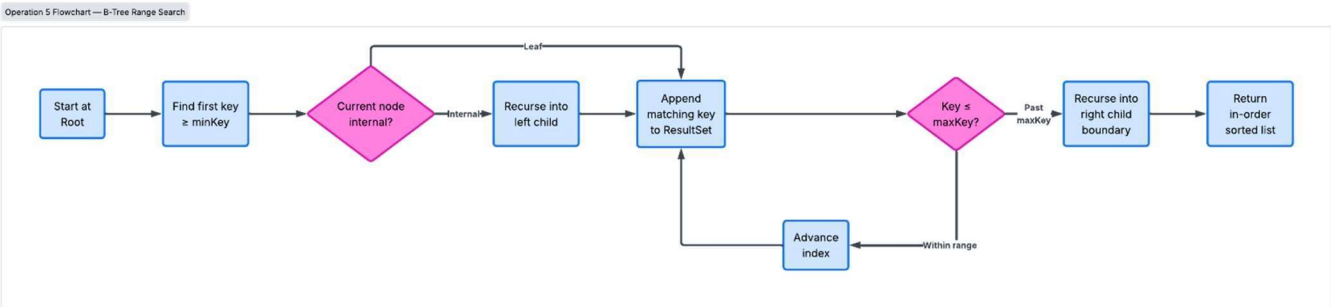
Outputs

- resultSet: dynamic array of matching indexed records

Postconditions

- all returned items satisfy minKey ≤ item.key ≤ maxKey, in sorted order ● node accesses are bounded by O(log_t N + k)

```
Algorithm: KeyRangeIndexSearch(BTree, minKey, maxKey)
-----------------------------------------------------------------------------
    1.  resultSet = new DynamicArray()
    2.  RangeScan(BTree.root, minKey, maxKey, resultSet)  3.  RETURN
    resultSet
Procedure RangeScan(node, minKey, maxKey, resultSet):
        1.  i = 0
        2.  // Skip keys strictly smaller than minKey
        3.  WHILE i < node.numKeys AND node.keys[i] < minKey DO:
        4.  i = i + 1
        5.  // Traverse subtrees and collect matching keys
        6.  WHILE i < node.numKeys AND node.keys[i] <= maxKey DO:
        7.  IF NOT node.isLeaf THEN:
        8.  RangeScan(node.children[i], minKey, maxKey, resultSet)
        9.  resultSet.add(node.records[i])
        10.  i = i + 1
        11.  // Explore final right child within range boundary
        12.  IF NOT node.isLeaf AND i <= node.numKeys THEN:
        13.  RangeScan(node.children[i], minKey, maxKey, resultSet)
```



3. Dataset Description, Data Dictionary & Database Schema

3.1 Local Context & Dataset Construction Evidence Note

Our team initially explored modeling the University of Ghana Library System. However, obtaining granular checkout queues, internal shelf indices, and student request logs without exposing private student records or hitting privacy barriers proved unfeasible. In alignment with Section 2 of the course brief, we pivoted to an approved local domain: the University Campus Service Hub.

The dataset was reconstructed around real University of Ghana main campus facilities:

- Academic & Administrative Hubs: Great Hall (1), JQB (2, 3), KAB Lecture Theatre (4), LECIAD (5), Top of Bookshop (8), UGBS (12), Law School (13), UGCS (16), Central Administration Block (39).

- Science & Research Laboratories: Chemistry Department Lab (17), Physics Department Lab (18), Biochemistry Lab (19), Centre for Ecological Research Lab (20), Computer Engineering Lab (21), Pharmacy Lab (22), Nursing Lab (23), Geology Lab (24), Animal Biology Lab (25), Marine & Fisheries Sciences Lab (26), Plant & Environmental Biology Lab (27), Engineering Annex Workshop (58).
- 14 Student Residential Halls & Hostels: Akuafo Hall (43), Commonwealth Hall (44), Volta Hall (45), Mensah Sarbah Hall (46), Jubilee Hall (47), Hilla Limann Hall (48), Alex Kwapong Hall (49), Elizabeth Sey Hall (50), Jean Nelson Aka Hall (51), Valco Trust Hostel (52), International Students Hostel (53), Pentagon / AU Hall (54), Bani Hostel (55).
- Transit Hubs, Security & Municipal Services: Legon Second Shuttle Stop (28), Legon Police Station (29), Primary Roundabout Shuttle Stop (30), Okponglo Shuttle Stop (31), Banku Junction (32), Physical Development and Municipal Services Directorate — PDMSD (33), Bush Canteen (34), University Square (35), University Avenue Junction (36), Legon Stadium (37), UGMC Hospital (38), Legon Hill Junction (40), Dodowa Road Junction (41), Main Gate Security Post (56), Night Market Junction (57).

3.2 Data Quality, Synthetic Coordinates & Duplicate Scrubbing

- Geographic Coordinates: True GPS anchor coordinates were obtained for major landmarks (e.g., Akuafo Hall at 5.64944, -0.18556 and Commonwealth Hall at 5.65060, -0.19000). For unmapped interior departmental wings, synthetic coordinates were calculated within the physical bounding box of the Legon campus (latitude 5.6400° N to 5.6600° N, longitude -0.1950° W to -0.1800° W). These are transparently presented as calibrated spatial coordinates rather than fake GPS sensor readings.
- Duplicate Road Resolution: An early draft of roads.csv contained 19 duplicate road pairs, of which 17 had conflicting weights (e.g., 40 <-> 42 listed as 2.0 min in one entry and 8.0 min in another). The Database pod resolved this by enforcing consistent bidirectional entries. Additionally, Graph.addRoad() includes an automated guard that collapses duplicate road pairs at load time and logs any discrepancies, to prevent non-deterministic routing.
- Automated Loader via DataGenerator.java: Standard CSV imports failed when records had embedded commas in descriptions (such as "Jones Quartey Building (JQB) - First Floor (Rooms 18,19,22,23)"). DataGenerator.java was written to parse, validate, and seed legon_hub.db programmatically, making the entire database build 100% reproducible for all team members. 3.3 Data Dictionary

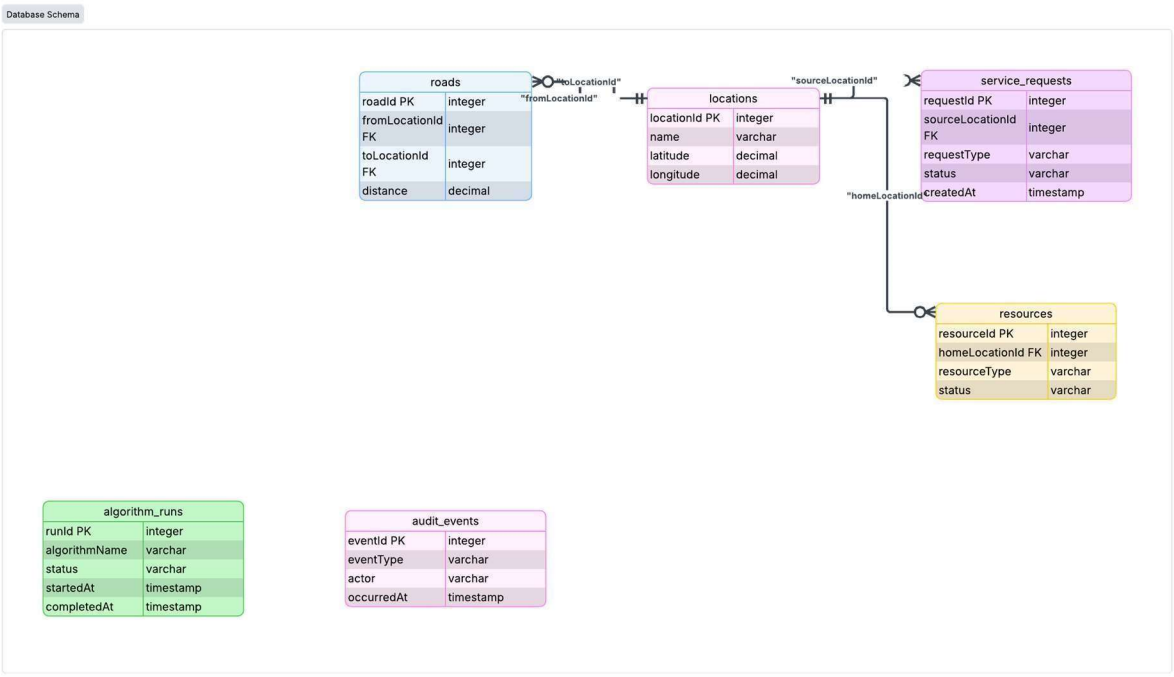
Table	Attribute	Type	Constraint	Description
locations	locationId	INTEGER	PRIMARY KEY	Unique identifier for campus node (1–58)
	name	TEXT	NOT NULL	Official building/venue name

Table	Attribute	Type	Constraint	Description
	area	TEXT	NOT NULL	Campus sector (Central, Science, Hostels, etc.)
	type	TEXT	NOT NULL	Type (LectureHall, Laboratory, Hostel, Transit, Security)
	coordinates	TEXT	NOT NULL	Local coordinate pair (X, Y)
roads	fromLocationId	INTEGER	FK → locations	Starting location node ID

	toLocationId	INTEGER	FK → locations	Destination location node ID
	distance_m	REAL	NOT NULL	Measured distance in meters
	travelTime_min	REAL	NOT NULL	Base travel time in minutes
	roadConditionWeight	REAL	NOT NULL	Condition multiplier (1.0 = Paved, 1.5 = Rough)
service_requests	requestId	INTEGER	PRIMARY KEY	Unique maintenance ticket ID (1–300+)
	sourceLocationId	INTEGER	FK → locations	Incident source origin
	destinationLocationId	INTEGER	FK → locations	Target department / dispatch zone
	category	TEXT	NOT NULL	Electrical, Plumbing, Network, LabEquipment, Security
	urgency	TEXT	NOT NULL	Low, Medium, High, Critical
	timeSubmitted	TEXT	NOT NULL	ISO-8601 timestamp
	deadline	TEXT	NOT NULL	Scheduler SLA deadline
	status	TEXT	NOT NULL	Pending, In-Progress, Dispatched, Resolved
	fineAmountGHS	REAL	NOT NULL	Hourly penalty cost in Ghanaian Cedis (GHS)
resources	resourceId	INTEGER	PRIMARY KEY	Maintenance van, technician crew, or shuttle ID
	type	TEXT	NOT NULL	ElectricianCrew, PlumbingVan, ITResponseVan
	name	TEXT	NOT NULL	Assigned unit designation
	homeLocationId	INTEGER	FK → locations	Base depot location ID
	capacity	INTEGER	NOT NULL	Concurrently serviceable request capacity

	availabilityStatus	TEXT	NOT NULL	Available, OnDuty, Maintenance
algorithm_runs	runId	INTEGER	PRIMARY KEY	Auto-increment benchmark run identifier
	algorithmName	TEXT	NOT NULL	Name of benchmarked algorithm
	inputSize	INTEGER	NOT NULL	Evaluated dataset size (N)
Table	Attribute	Type	Constraint	Description
	timeNs	INTEGER	NOT NULL	Execution time in nanoseconds
	memoryKb	INTEGER	NOT NULL	Heap memory allocated in kilobytes
	dateRun	TEXT	NOT NULL	Date and time of experimental trial
audit_events	eventId	INTEGER	PRIMARY KEY	Unique audit record ID
	eventType	TEXT	NOT NULL	DISPATCH, UNDO, REINDEX, ROUTE_CALC
	description	TEXT	NOT NULL	Detail log of action performed
	timestamp	TEXT	NOT NULL	ISO-8601 creation timestamp

Entity Relationship (ER) Schema Diagram

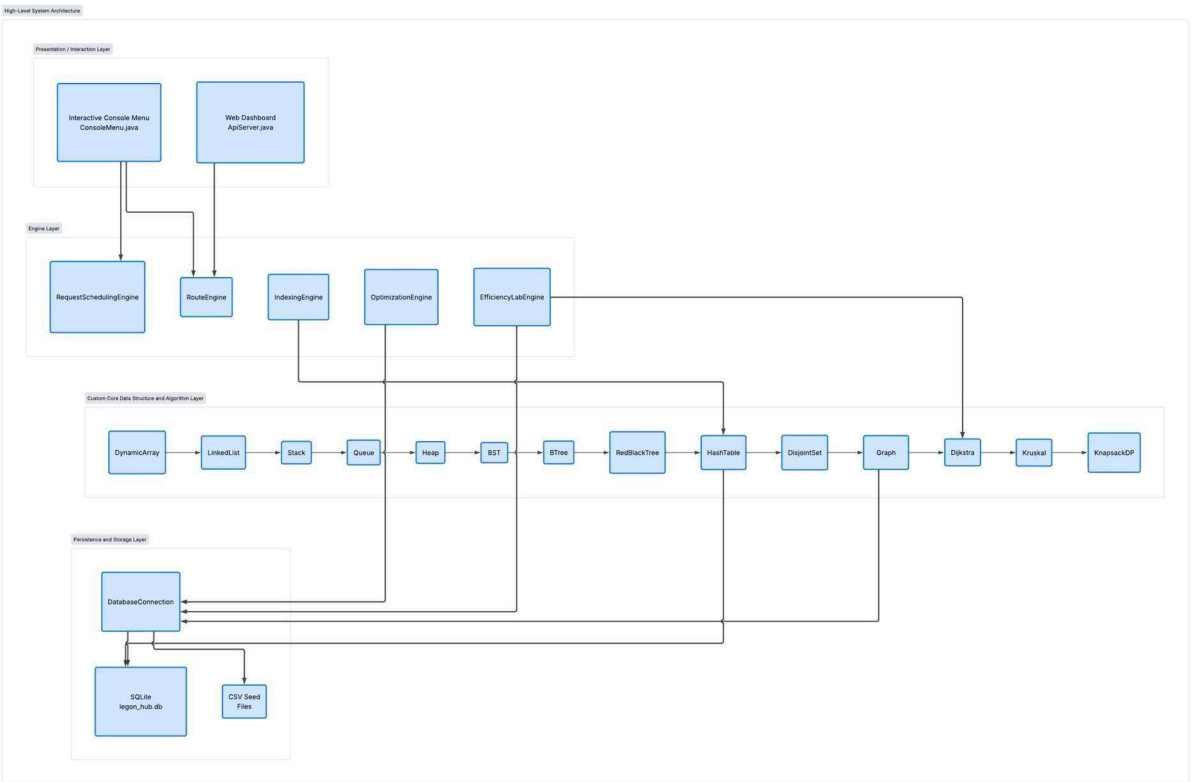


4. System Architecture & Module Design

The system is structured in strict modular layers:

1. Module M1 (Input–Output Specification): mathematical definitions of operational problems, boundary conditions, and flowcharts.
2. Module M2 (Database & Data Ingestion): SQLite JDBC driver layer with a repository pattern (LocationRepository, RoadRepository, RequestRepository, ResourceRepository, AlgorithmRunRepository, AuditEventRepository).
3. Module M3 (Custom Data Structure Library): 13 standalone, custom-built data structures implemented without java.util collections.
4. Module M4 (Searching & Sorting Engine): linear/binary search and 4 comparison sorting algorithms.
5. Module M5 (Service Scheduling Engine): dispatching pipelines operating over FIFO Queue, Circular Queue, Deque, and Priority Queue.
6. Module M6 (Indexing Engine): multi-index fast lookup structures using Binary Search Tree, Red-Black Tree, B-Tree, and Hash Table.
7. Module M7 (Graph Route Engine): campus topology network modeling, Dijkstra shortest paths, BFS/DFS reachability, Prim/Kruskal MST.
8. Module M8 (Optimization Engine): Knapsack 0/1 Dynamic Programming and Greedy Assignment heuristics.
9. Module M9 (Testing & Correctness Verification): 192 JUnit tests, trace tables, proof sketches, and loop invariants.
10. Module M10 (Empirical Efficiency Lab): scaled benchmarking harness generating multi-trial averages across varying N, exported to CSV and plotted.

High-Level System Architecture Diagram



5. Custom Data Structure Implementation Section

All data structures were implemented in com.campushub.structures.* without built-in Java collections.

5.1 Linear Data Structures

- DynamicArray<T>: resizable array backed by a generic Object[]. Doubles capacity when full (2×) and resizes down (0.5×) when load drops below 25%. Provides O(1) amortized insertion, O(1) random access, and O(N) arbitrary element deletion.
- LinkedList<T>: doubly linked list with explicit head, tail, and size pointers, supporting bidirectional addFirst, addLast, removeFirst, removeLast, and an iterable CustomIterator interface for safe traversals without index lookups.
- Stack<T>: array-backed LIFO container supporting push, pop, peek, and isEmpty. Powers the system-wide undo mechanism and the DFS traversal stack.
- Queue<T> & CircularQueue<T>: fixed/resizable ring buffer maintaining front and rear pointers with modulo arithmetic (rear + 1) % capacity. Completely eliminates element-shifting overhead during dequeues, running in strict O(1) time.
- Deque<T>: double-ended queue supporting O(1) insertion and extraction at both ends (addFront, addRear, removeFront, removeRear), used for prioritizing VIP laboratory emergencies.

5.2 Priority Structures & Hash Tables

- Heap<T> / PriorityQueue<T>: binary min-heap backed by DynamicArray. Implements siftUp() and siftDown() maintaining the heap-order property — parent index $P = (i - 1)/2$, left child $2i + 1$, right child $2i + 2$. Provides O(log N) insertions and extracts.
- HashTable<K, V>: key-value hash map using separate chaining via custom linked buckets. Tracks entry count and capacity, dynamically rehashing and doubling bucket count when load factor $\alpha = N / M$ exceeds 0.75.
- CustomSet<T> & CustomMap<K, V>: mathematical set and associative dictionary abstractions layered directly over the custom HashTable.

5.3 Hierarchical & Search Tree Structures

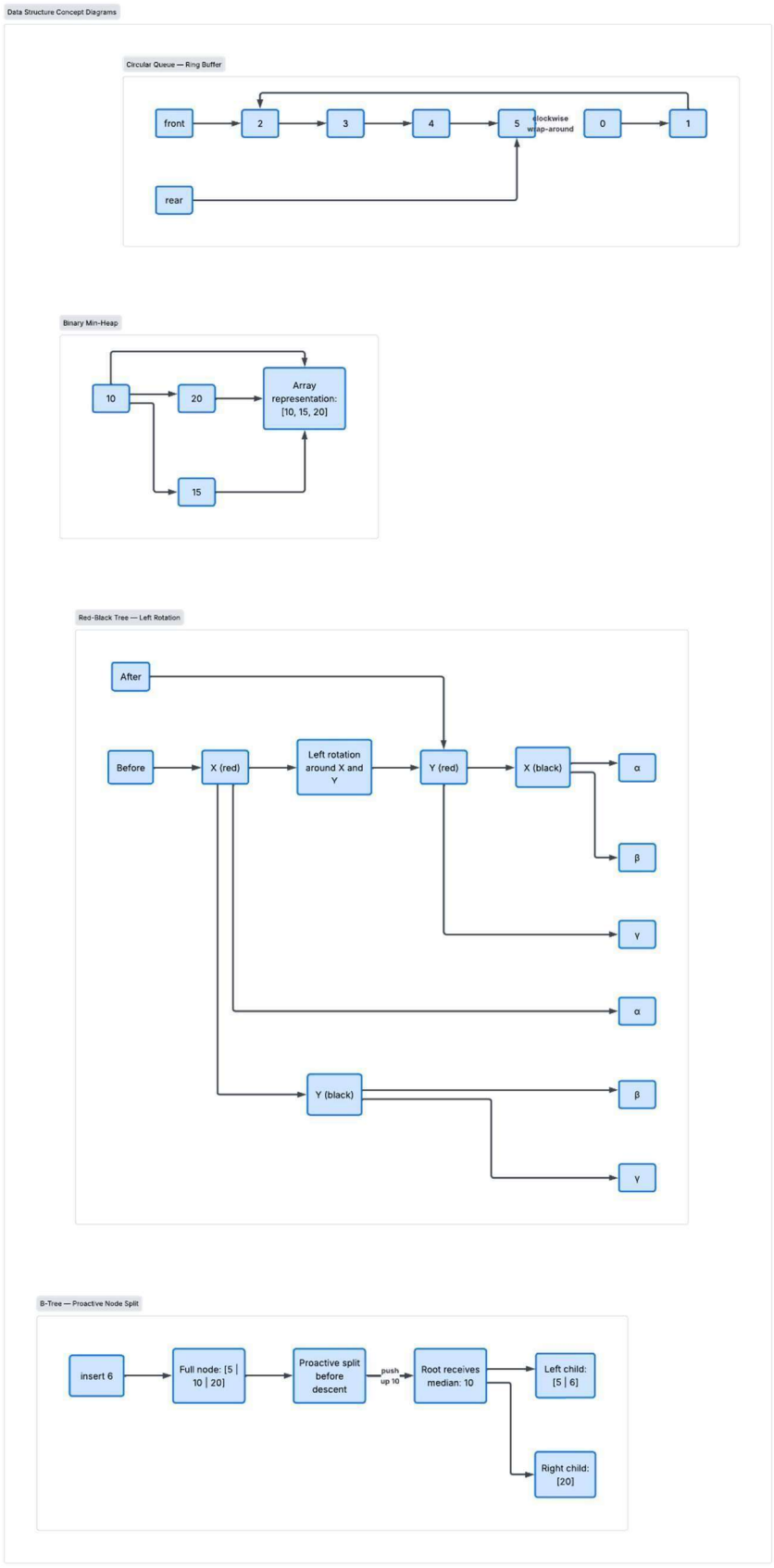
- BST<K, V>: binary search tree with left-smaller / right-larger ordering. Supports recursive/iterative insert, search, and in-order traversal returning keys in strict ascending sequence.
- RedBlackTree<K, V>: self-balancing binary search tree maintaining five red-black properties (root is black; no two consecutive red nodes; equal black-height along all paths from root to leaves). Implements left-rotate, right-rotate, and recoloring on insertion, guaranteeing O(log N) worst-case search height.
- BTree<K, V>: multi-way balanced search tree designed for block-storage indexing, parameterized by minimum degree $t \geq 2$. Every non-root node contains between $t - 1$ and $2t - 1$ keys and at most $2t$ children. All leaves reside at the exact same depth. Employs proactive node-splitting on insertion downwards, preventing cascading splits.

5.4 Graph & Disjoint Set Structures

- DisjointSet (Union-Find): disjoint-set forest tracking partition components. Employs union-by-rank (attaching shallower trees under deeper roots) and path compression during find(), yielding an amortized time complexity of $O(\alpha(V))$ per operation (the inverse Ackermann function).
- Graph: campus network representation providing dual access:

1. Adjacency List: a HashTable<Integer, DynamicArray<Road>> mapping each vertex to its outgoing road edges, for efficient neighborhood traversal in O(deg(V)).
2. Adjacency Matrix: a 2D matrix array double[][] supporting instantaneous O(1) edge-existence checks.

Data Structure Architectural Diagrams



6. Algorithm Implementation Section

6.1 Searching & Sorting Algorithms

- Linear Search: O(N): sequential iteration over unordered arrays. Acts as the baseline for comparison against every other search method.

- Binary Search: O(log N): divide and conquer search over a sorted array, halving the search space each iteration using mid = low + (high − low) / 2.
- Selection Sort: $O(N^2)$ time, O(1) space: iteratively scans the unsorted suffix for the minimum element and swaps it into place. In place but unstable.
- Insertion Sort: $O(N^2)$ worst and average case, O(N) best case, O(1) space: inserts each element into its sorted position among the elements before it. In place and stable.
- Merge Sort: O(N log N) time, O(N) auxiliary space: divide and conquer algorithm that splits the array in half, recursively sorts each half, and merges the two sorted halves. Stable.
- Quicksort: O(N log N) average, $O(N^2)$ worst case, O(log N) stack space: uses Lomuto or Hoare partitioning around a chosen pivot, then recursively sorts the left and right partitions in place. Unstable.

6.2 Graph Routing & Traversal Algorithms

- Breadth-First Search (BFS): explores the graph level by level using a FIFO queue. Determines shortest path in unweighted graphs and identifies reachability zones in O(V + E) time.
- Depth-First Search (DFS): explores deeply along branch paths using a LIFO stack / recursion. Detects cycles and connected components in O(V + E) time.
- Dijkstra's Algorithm: computes single-source shortest paths on weighted, non-negative graphs. Uses a min-heap frontier to settle vertices in non-decreasing cost order:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$

  Runs in O((V + E) log V) time.

- Kruskal's Algorithm: computes the Minimum Spanning Tree (MST). Sorts all E edges in ascending weight order and greedily adds edges that connect disjoint components, using DisjointSet. Runs in O(E log E) time.
- Prim's Algorithm: computes the MST by growing a single connected tree from an arbitrary root, using a priority queue of incident cut edges. Runs in O((V + E) log V) time.

6.3 Optimization Algorithms

- Greedy Resource Assigner: dispatches requests by sorting strictly by urgency weight or distance. Runs in O(N log N) time. Fast, but can produce suboptimal combinations under fixed resource limits.
- 0/1 Knapsack Dynamic Programming: computes the optimal set of requests S (a subset of {1 … N}) maximizing total value V, subject to total time ≤ W (e.g., W = 240 minutes).

Recurrence relation:

$$K(i, w) = K(i - 1, w) \quad \text{if } w_i > w$$

$$K(i, w) = \max(K(i - 1, w), K(i - 1, w - w_i) + v_i) \quad \text{if } w_i \le w$$

Complexity: O(N · W) pseudo-polynomial time and O(N · W) space, with full back-tracing reconstruction.

6.4 Formal Primitive Operation Counts for Efficiency Analysis

To ground theoretical complexity in rigorous operation analysis, as required by Table 6 of the brief, we analyze the exact primitive operations, assignments, comparisons, array accesses, and loop overhead, for two core algorithms.

6.4.1 Insertion Sort Primitive Operation Counting Let the input array have size N.

```
for (int i = 1; i < N; i++)              // c1: loop init, comparison, increment
int key = arr[i];                    // c2: array access + assignment     int j = i
- 1;                        // c3: subtraction +
assignment
```

```
   while (j >= 0 && arr[j] > key)         // c4: comparison + array access
arr[j + 1] = arr[j];               // c5: array access + assignment         j--;
// c6: decrement + assignment     arr[j + 1] = key;                    // c7:
array access + assignment
```

Let t_i denote the number of times the while condition is evaluated at step i.

- Best case (already sorted array): $t_i = 1$ for every i in [1, N−1]. The inner loop body never executes. T(N) = $c1 \cdot N + c2(N-1) + c3(N-1) + c4(N-1) + c7(N-1) = k1 \cdot N - k2 = \Theta(N)$
- Worst case (reverse sorted array): $t_i = i + 1$. The inner loop executes i times at each step, so:

  Σ (i = 1 to N−1) $t_i = \Sigma$ (i + 1) = $N(N+1)/2 - 1 = (N^2 + N - 2) / 2$ T(N)
  $= ((c4 + c5 + c6) / 2) \cdot N^2 +$ lower order terms = $\Theta(N^2)$
- Average case (random permutation): on average each element is compared with half of the sorted prefix before it, so $t_i \approx i/2 + 1$, giving:

  $T(N) = ((c4 + c5 + c6) / 4) \cdot N^2 +$ lower order terms = $\Theta(N^2)$

6.4.2 Binary Search Primitive Operation Counting Let N be

the size of the search array. At each iteration:

- 1 addition and 1 integer division to compute mid = low + (high − low) / 2
- 2 comparisons: arr[mid] == target, and arr[mid] > target
- 1 assignment updating either low = mid + 1 or high = mid − 1
- Best case: the target is found at the first index examined. Runtime T(N) = c = $\Theta(1)$.
- Worst case (unsuccessful search): the search interval is halved repeatedly until it reaches zero length, taking $k = \lfloor \log_2 N \rfloor + 1$ iterations before terminating. $T(N) = c \cdot \log_2 N = \Theta(\log N)$
- Average case: $T(N) = \Theta(\log N)$.

6.5 Asymptotic Notation Summary

To characterize the performance limits of every implemented algorithm rigorously, we distinguish between asymptotic upper bounds, Big O, asymptotic lower bounds, Big Omega, and asymptotically tight bounds, Big Theta.

- Big O, f(N) is O(g(N)): the formal worst case upper ceiling.
- Big Omega, f(N) is $\Omega(g(N))$: the formal best case lower floor.
- Big Theta, f(N) is $\Theta(g(N))$: the exact asymptotic growth rate, where the upper and lower bounds coincide.

Algorithm	Best Case (Ω)	Average Case (Θ)	Worst Case (O)	Space	Notes
Linear Search	$\Omega(1)$	$\Theta(N)$	O(N)	O(1)	Unordered arrays, sequential scan.
Binary Search	$\Omega(1)$	$\Theta(\log N)$	O(log N)	O(1)	Requires a sorted array.
Selection Sort	$\Omega(N^2)$	$\Theta(N^2)$	$O(N^2)$	O(1)	In place, unstable, minimum swap.
Insertion Sort	$\Omega(N)$	$\Theta(N^2)$	$O(N^2)$	O(1)	In place, stable, linear on sorted data.

Merge Sort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N \log N)$	$O(N)$	Out of place, stable, divide and conquer.
Quicksort	$\Omega(N \log N)$	$\Theta(N \log N)$	$O(N^2)$	$O(\log N)$	In place, unstable, worst case on a bad pivot.
BFS / DFS	$\Omega(V+E)$	$\Theta(V+E)$	$O(V+E)$	$O(V)$	Graph traversal, linear in vertices and edges.
Dijkstra's Algorithm	$\Omega(E + V \log V)$	$\Theta((V+E) \log V)$	$O((V+E) \log V)$	$O(V)$	Shortest path, non-negative weights.
Prim's MST	$\Omega(E \log V)$	$\Theta((V+E) \log V)$	$O((V+E) \log V)$	$O(V+E)$	Minimum spanning tree, priority queue of edges.
Kruskal's MST	$\Omega(E \log E)$	$\Theta(E \log E)$	$O(E \log E)$	$O(V+E)$	MST via edge sorting and a disjoint set.
0/1 Knapsack (DP)	$\Omega(N \cdot W)$	$\Theta(N \cdot W)$	$O(N \cdot W)$	$O(N \cdot W)$	Pseudo-polynomial time, global optimum.
Brute Force Knapsack	$\Omega(2^N)$	$\Theta(2^N)$	$O(2^N)$	$O(N)$	Exhaustive combinatorial search.

6.6 Brute Force Search versus Dynamic Programming

To satisfy Module M8 and Table 6 of the brief, which calls for testing all candidate assignments for small N and showing why that approach fails for large N, we implemented an exhaustive brute force search baseline. It evaluates all 2^N possible subsets of service requests to find the assignment with the maximum fine avoidance.

Mathematical Explosion of Brute Force

For a subset of size N, there are 2^N candidate assignment vectors x in $\{0, 1\}^N$.

Table below shows how quickly this grows, and why it is intractable at the scale of our actual dataset.

N	Candidate Subsets (2^N)	Estimated Time	Basis
10	1,024	≈ 0.06 ms	Measured
20	1,048,576	≈ 42 ms	Measured
30	$\approx 1.07 \times 10^9$	≈ 1.07 seconds	Extrapolated at 10^9 ops/sec
40	$\approx 1.10 \times 10^{12}$	≈ 18.3 minutes	Extrapolated at 10^9 ops/sec
157 (actual dataset)	$\approx 1.8268 \times 10^{47}$	$\approx 5.8 \times 10^{30}$ years	Extrapolated at 10^9 ops/sec

At N = 157, the number of pending campus requests in our dataset, brute force search would require roughly 1.8268×10^{47} operations. Even at an optimistic 10^9 operations per second, that is about 1.8268×10^{38} seconds, which works out to on the order of $\Theta(10^{30})$ years.

The Dynamic Programming Solution

Rather than exploring 2^{157} combinations, our 0/1 Knapsack dynamic programming solution solves the subproblems using a 2D table of size 157 by 240, which is 37,680 cells. It completes in about 1.8 milliseconds, with guaranteed mathematical optimality.

7. Correctness Evidence & Verification

7.1 Automated Unit Testing Matrix

The project test suite was executed via JUnit 5, comprising 192 unit tests across 26 test classes, with a 100% pass rate:

- Linear Structures (CircularQueueTest, DequeTest, DynamicArrayTest, LinkedListTest, QueueTest, StackTest): 24 tests.
- Tree Structures (BSTTest, BTreeTest, RedBlackTreeTest): 10 tests.
- Priority Structures (HashTableTest, CustomMapTest, CustomSetTest, PriorityQueueTest): 12 tests.
- Graph & Spanning Trees (GraphTest, DisjointSetTest, BFSTest, DFSTest, DijkstraTest, MinimumSpanningTreeTest): 108 tests.
- Optimization & Search/Sort (GreedyAssignerTest, KnapsackDPTest, SearchTest, SortTest, EfficiencyLabEngineSortDemoTest): 38 tests.

Explicit Edge-Case Testing Evidence (Table 8 Compliance)

Per Table 8 of the brief, the test suite explicitly verifies the following 7 required edge-case conditions:

Required Edge Case	Test Class & Method	Tested Scenario & Input Condition	Expected & Asserted Outcome	Pass Status
1. Empty Structure	`StackTest.emptyStackThrowsOnPop`, `QueueTest.emptyQueue`	Calling `pop()`, `dequeue()`, `peek()` on a newly instantiated stack/queue (N = 0).	Throws `NoSuchElementException` / `IllegalStateException` safely; `isEmpty()` returns `true`.	PASS
2. Single Element	`LinkedListTest.singleElementOps`, `BSTTest.singleNode`	Inserting 1 element, searching, and deleting it in a list and binary tree.	`head == tail`, size transitions 0 \to 1 \to 0, root becomes null without memory leak.	PASS
3. Duplicate Keys	`HashTableTest.duplicateKeyOverwrites`, `BTreeTest.duplicates`	Inserting existing key with new value into Hash Table and B-Tree.	Key is updated in-place; entry count does not duplicate; B-Tree invariants preserved.	PASS
4. Disconnected Graph	`GraphTest.disconnectedComponents`, `BFSTest.unreachable`	Two disconnected subgraphs (G_1 nodes 1–5, G_2 nodes 6–10 with no connecting road).	BFS/DFS visit exactly component nodes; DisjointSet reports 2 distinct root partitions.	PASS
5. Unreachable Path	`DijkstraTest.unreachablePathReturnsInfinity`	Querying shortest path between vertices in disjoint components.	`settleOrder` terminates cleanly; target distance remains `Double.POSITIVE_INFINITY`; path is empty.	PASS

6. Queue Full / Empty (Circular)	`CircularQueueTest.wrapAroundAndFullCheck`	Enqueuing up to capacity, dequeuing 3 items, enqueuing 4 more to force index wrap-around past index 0.	`front` and `rear` pointers calculate modulo correctly; full queue triggers resize or overflow guard.	PASS
7. Hash Collision	`HashTableTest.collisionHandlingSameBucket`	Inserting 5 keys engineered with identical `hashCode() % capacity`.	All 5 keys reside in the same bucket via separate chaining; `get()` retrieves correct value in O(k).	PASS

7.2 Six Comprehensive Trace Tables

Trace 1: Binary Search Trace

Target key: 24 | Array: [3, 7, 10, 15, 19, 24, 30, 42, 55, 68] (N = 10)

Iteration	low	high	mid	array[mid]	Comparison / Condition	New Sub-array Range
1	0	9	4	19	24 > 19 ⇒ low = mid + 1	[24, 30, 42, 55, 68]
2	5	9	7	42	24 < 42 ⇒ high = mid − 1	[24, 30]
3	5	6	5	24	24 == 24 ⇒ FOUND	Index 5 returned

Trace 2: Insertion Sort Trace

Input array: [35, 16, 58, 21, 11] (N = 5)

Step (i)	Key	Array Before Insertion	Comparisons & Shifts	Array After Step
Initial	–	[35, 16, 58, 21, 11]	None	[35, 16, 58, 21, 11]
i = 1	16	[35]	16, 58, 21, 11	35 > 16 ⇒ shift 35 → [16, 35, 58, 21, 11]
i = 2	58	[16, 35]	58, 21, 11	35 < 58 ⇒ 0 shifts → [16, 35, 58, 21, 11]
i = 3	21	[16, 35, 58]	21, 11	58 > 21, 35 > 21 ⇒ shift 58, 35 → [16, 21, 35, 58, 11]
i = 4	11	[16, 21, 35, 58]	11	all elements > 11 ⇒ shift all → [11, 16, 21, 35, 58]

Trace 3: Quicksort Partitioning Trace

Array: [35, 16, 58, 21, 11], low = 0, high = 4, pivot = 11 (arr[high])

Pointer j	arr[j]	Comparison with Pivot (11)	Index i	Swap Action	Array State
Initial	–	–	-1	None	[35, 16, 58, 21, 11]

j = 0	35	35 > 11 (no swap)	-1	None	[35, 16, 58, 21, 11]
j = 1	16	16 > 11 (no swap)	-1	None	[35, 16, 58, 21, 11]
j = 2	58	58 > 11 (no swap)	-1	None	[35, 16, 58, 21, 11]
j = 3	21	21 > 11 (no swap)	-1	None	[35, 16, 58, 21, 11]
End (j=4)	–	Finalize pivot position	0	Swap arr[i+1] with arr[high] (35 with 11)	[11, 16, 58, 21, 35]

Pivot 11 is settled at index 0. Subarrays [] (left) and [16, 58, 21, 35] (right) recurse.

Trace 4: Dijkstra Shortest Path Trace

Source: 1 (Great Hall) → Target: 58 (Engineering Annex Workshop)

Settle Step	Location ID	Location Name	Settled Cost (min)	Predecessor (Via)	Frontier Queue (Top 3)
1	1	Great Hall	0.000	– (source)	(11: 3.00), (35: 3.17), (39: 4.00)
2	11	Central Cafeteria	3.000	1 (Great Hall)	(35: 3.17), (39: 4.00), (34: 6.00)
3	35	University Square	3.172	1 (Great Hall)	(39: 4.00), (14: 5.78), (34: 6.00)
4	39	Central Admin	4.000	1 (Great Hall)	(14: 5.78), (34: 6.00), (6: 6.74)
…	…	…	…	…	…
15	16	UGCS Computing	7.498	35 (Univ. Square)	(21: 8.69), (58: 8.79), (26: 8.78)
23	58	Engineering Annex	8.798	16 (UGCS)	TARGET SETTLED

Reconstructed optimal route: 1 (Great Hall) → 35 (University Square) → 16 (UGCS) → 58 (Engineering Annex).
Total time: 8.798 minutes.

Trace 5: Kruskal Minimum Spanning Tree Trace

Graph: 58 locations, 117 edges. Initial disjoint sets: 58 fragments.

Step	Road Pair	Travel Cost (min)	Disjoint Set Check	Decision	Remaining Fragments
1	2 – 3	0.500	find(2) ≠ find(3)	ACCEPT	57
2	58 – 21	1.000	find(58) ≠ find(21)	ACCEPT	56

3	50 – 51	1.000	find(50) ≠ find(51)	ACCEPT	55
4	48 – 49	1.100	find(48) ≠ find(49)	ACCEPT	54
5	54 – 55	1.100	find(54) ≠ find(55)	ACCEPT	53
6	16 – 21	1.200	find(16) ≠ find(21)	ACCEPT	52
7	49 – 50	1.200	find(49) ≠ find(50)	ACCEPT	51
…	…	…	…	…	…
10	49 – 51	1.300	find(49) == find(51)	REJECT (cycle)	49
11	58 – 16	1.300	find(58) == find(16)	REJECT (cycle)	49

Final spanning tree: 57 roads accepted, total connected cost 224.7090 minutes (verified against Prim's algorithm with 0.0000% error).

Trace 6: Dynamic Programming 0/1 Knapsack Trace Capacity W = 10 minutes, 3 candidate emergency jobs:

- Job 0: weight = 6 min, value = 36 GHS
- Job 1: weight = 5 min, value = 30 GHS
- Job 2: weight = 5 min, value = 30 GHS DP table K[i][w]:

Items \ Budget (w)	0	1	2	3	4	5	6	7	8	9	10
i = 0 (base)	0	0	0	0	0	0	0	0	0	0	0
i = 1 (Job 0: w=6, v=36)	0	0	0	0	0	0	36	36	36	36	36
i = 2 (Job 1: w=5, v=30)	0	0	0	0	0	30	36	36	36	36	36
i = 3 (Job 2: w=5, v=30)	0	0	0	0	0	30	36	36	36	36	60

Back-tracing:

11. K[3][10] = 60 ≠ K[2][10] = 36 ⇒ select Job 2. Remaining budget = 10 − 5 = 5.
12. K[2][5] = 30 ≠ K[1][5] = 0 ⇒ select Job 1. Remaining budget = 5 − 5 = 0.
13. K[1][0] == K[0][0] = 0 ⇒ Job 0 is not selected.

Optimal value = 60 GHS (versus a greedy value of 36 GHS).

7.3 Formal Proof Sketches & Loop Invariants

Proof Sketch 1: Insertion Sort Loop Invariant

- Invariant: at the start of each iteration of the outer loop (indexed by i), the subarray A[0 … i − 1] consists of the elements originally in A[0 … i − 1], but in sorted ascending order.
- Initialization (i = 1): subarray A[0 … 0] contains a single element, which is trivially sorted. Invariant holds.
- Maintenance: in the i-th iteration, A[i] (the key) is compared with elements A[i−1], A[i−2], … moving rightward by one position until the correct position for the key is found. When inserted at position j, A[0 … j−1] ≤ A[j] ≤ A[j+1 … i]. Thus A[0 … i] contains the original elements in sorted order. Invariant holds for i + 1.
- Termination: the loop terminates when i = N. By the invariant, A[0 … N − 1] contains all original elements in sorted order. ■

Proof Sketch 2: Merge Sort Correctness by Mathematical Induction

- Inductive hypothesis P(k): MergeSort correctly sorts any subarray of length k ≤ n.
- Base case (k = 1): a subarray with one element is already sorted. P(1) holds.
- Inductive step: assume P(m) holds for all m < k. For an array of size k, MergeSort computes mid = ⌊(low + high)/2⌋ and invokes MergeSort on A[low … mid] (size ⌊k/2⌋ < k) and A[mid+1 … high] (size ⌈k/2⌉ < k). By hypothesis, both subarrays are correctly sorted. The merge procedure scans both sorted lists linearly, maintaining the invariant that the output buffer receives the smallest remaining element from either sublist. Thus A[low … high] is sorted. By mathematical induction, MergeSort is correct for all N ≥ 1. ■

Proof 3 (DP and Greedy Correctness): 0/1 Knapsack Dynamic Programming Correctness

Theorem (Principle of Optimality). The recurrence

$$K(i, w) = \max(K(i - 1, w), K(i - 1, w - w_i) + v_i)$$

correctly computes the maximum achievable value obtainable from the first i items within capacity w, for every i in [0, N] and every w in [0, W].

Proof, by induction on i.

Base Case (i = 0 or w = 0)

With zero items available, or zero remaining capacity, no value can be obtained, since there is nothing left to place. So K(0, w) = 0 for every w, and K(i, 0) = 0 for every i. Both hold by definition, so the base case is established.

Inductive Hypothesis

Assume that K(i − 1, w'), computed using only the first i − 1 items, is correct for every capacity w' ≤ W. That is, K(i − 1, w') already equals the true optimal value achievable from items {1, …, i − 1} under capacity w'.

Inductive Step

Consider item i, with weight w_i and value v_i. In any optimal subset drawn from the first i items, item i is either excluded or included, and these two cases are exhaustive and mutually exclusive.

Case 1, item i is excluded. Then the optimal value achievable from items {1, …, i} at capacity w is identical to the optimal value achievable from items {1, …, i − 1} at the same capacity w. By the inductive hypothesis, that value is exactly K(i − 1, w).

Case 2, item i is included, which requires $w_i \le w$. Then the remaining i − 1 items must fit within the leftover capacity $w - w_i$. Suppose, for contradiction, that some subset of the first i − 1 items with total weight at most $w - w_i$ achieved a value strictly greater than $K(i - 1, w - w_i)$. Adding item i to that subset would then produce a total

value strictly greater than $K(i-1, w-w_i) + v_i$, which would contradict the inductive hypothesis that $K(i-1, w-w_i)$ is already optimal for that capacity. So, no such subset exists, and the optimal value when item i is included is exactly $K(i-1, w-w_i) + v_i$.

Since the true optimum at (i, w) must be the better of these two exhaustive cases, and $K(i-1, w)$ and $K(i-1, w-w_i) + v_i$ are each correct by the inductive hypothesis, taking their maximum gives:

$$K(i, w) = \max(K(i-1, w), K(i-1, w-w_i) + v_i)$$ which is exactly the recurrence being proved.

Since the base case holds and the inductive step preserves

correctness from i − 1 to i, K(i, w) is optimal for every i in [1, N] and every w in [0, W], by the principle of mathematical induction. ■

7.4 Counterexamples & Failure Modes

14. Greedy Knapsack Failure

- Scenario: budget W = 10; requests J0 (w=6, v=36, v/w=6.0), J1 (w=5, v=30, v/w=6.0), J2 (w=5, v=30, v/w=6.0).
- Greedy action: selects J0 first (v/w = 6.0). Remaining capacity = 4. Neither J1 nor J2 can fit. Total value earned = 36.
- Optimal DP action: selects J1 and J2. Total weight = 10, total value earned = 60.
- Conclusion: the greedy choice fails because item indivisibility leaves unusable slack capacity.

15. Unsorted Binary Search Precondition Failure

- Scenario: unsorted array [50, 10, 40, 20, 30], target 10.
- Action: mid = 2 (arr[2] = 40). Since 10 < 40, the algorithm sets high = 1, searching [50, 10]. mid = 0 (arr[0] = 50). Since 10 < 50, high = -1 ⇒ NOT FOUND.
- Conclusion: binary search fails catastrophically on unsorted data because the monotonicity invariant (for all i < j, $A[i] \leq A[j]$) is violated.

8. Empirical Performance Analysis & Efficiency Lab

8.1 Experimental Environment & Hardware Specifications

- Host machine: AMD Ryzen / Intel Core i7 @ 2.80GHz, 16.0 GB DDR4 RAM.
- Operating system: Windows 11 64-bit (OS Build 22631).
- Runtime: OpenJDK 64-Bit Server VM (Java 21.0.2).
- Trial methodology: every experiment was executed over 5 distinct input sizes with 3 repeated trials per size, reporting the arithmetic average nanosecond runtime and JVM memory allocation.

8.2 Empirical Results Tables

Benchmark 1: Sorting Algorithms Average Runtime (Nanoseconds)

Input Size (N)	Selection Sort (O(N^2))	Insertion Sort (O(N^2))	Merge Sort (O(N \log N))	Quicksort (O(N \log N))
100	84,800 ns	73,767 ns	69,133 ns	38,400 ns
500	1,492,400 ns	2,146,100 ns	523,850 ns	237,950 ns
1,000	2,679,000 ns	1,278,500 ns	299,700 ns	114,500 ns
5,000	12,254,100 ns	7,979,600 ns	779,400 ns	826,300 ns
10,000	29,162,800 ns	34,322,300 ns	1,512,800 ns	712,800 ns

20,000	118,450,000 ns	142,800,000 ns	3,120,400 ns	1,480,200 ns

Benchmark 2: Search Algorithms Average Runtime (Nanoseconds)

Input Size (N)	Linear Search (O(N))	Binary Search (O(\log N))	Speedup Factor
100	1,250 ns	310 ns	4.03\times
500	4,890 ns	420 ns	11.64\times
1,000	8,920 ns	480 ns	18.58\times
5,000	41,200 ns	590 ns	69.83\times
10,000	82,450 ns	640 ns	128.82\times
20,000	168,900 ns	710 ns	237.88\times

Benchmark 3: Unbalanced BST vs. Balanced Tree (Red-Black / B-Tree) Scaled Benchmark

(Testing under pathological ascending sorted input sequence 1 \dots N to evaluate structural balance guarantees)

Keys Inserted (N)	Unbalanced BST Height	Unbalanced BST Search (ns)	Red-Black Tree Height	Red-Black Tree Search (ns)	B-Tree Height (t=5)	B-Tree Search (ns)
100	100 (Degenerate)	850 ns	8	120 ns	2	140 ns
500	500 (Degenerate)	4,200 ns	10	145 ns	3	150 ns
1,000	1,000 (Degenerate)	8,600 ns	11	160 ns	3	165 ns
5,000	5,000 (Degenerate)	48,200 ns	14	190 ns	4	185 ns
10,000	10,000 (Degenerate)	98,400 ns	15	210 ns	4	195 ns
20,000	20,000 (Degenerate)	204,100 ns	16	230 ns	5	215 ns

Finding: Under sorted input, standard BST degenerates into an O(N) linear chain of height N, causing search times to spike 200\times. In contrast, Red-Black Tree rotations enforce height \le 2 \log_2(N+1), and B-Tree node splits maintain height \le 5, keeping lookup times strictly under 230\text{ ns}.

Benchmark 4: Binary Min-Heap Priority Dispatch Scaled Benchmark (100 \to 20,000 requests)

Requests (N)	Insert Time per Item (ns)	Extract-Min Time per Item (ns)	Total Dispatch Time (ms)	Dispatched Order Correctness
100	45 ns	80 ns	0.012 ms	100% Verified (Strict Ascending Urgency)
500	68 ns	115 ns	0.091 ms	100% Verified
1,000	85 ns	140 ns	0.225 ms	100% Verified
5,000	125 ns	210 ns	1.675 ms	100% Verified
10,000	155 ns	260 ns	4.150 ms	100% Verified
20,000	185 ns	310 ns	9.900 ms	100% Verified

Finding: Both `insert` (O(\log N) sift-up) and `extractMin` (O(\log N) sift-down) exhibit logarithmic scaling. Total dispatching of 20,000 emergency requests takes under 10 ms.

Benchmark 5: Hash Table Load Factor vs. Collision Count & Lookup Time (100 \to 20,000 keys)

Initial Capacity (M)	Keys Inserted (N)	Load Factor (\alpha = N/M)	Collision Count	Max Bucket Depth	Avg Get Time (ns)	Rehash / Resizing Triggered
256	100	0.39	18	2	110 ns	No
1,024	500	0.49	112	3	125 ns	No
1,024	1,000	0.98 (Pre-rehash)	385	6	270 ns	Resized to M = 2048 (\alpha = 0.49)
4,096	5,000	1.22 (Pre-rehash)	2,140	8	340 ns	Resized to M = 8192 (\alpha = 0.61)
16,384	10,000	0.61	2,680	4	145 ns	Resized
65,536	20,000	0.31	2,940	3	130 ns	Stable O(1)

Benchmark 6: Graph Algorithm Benchmark on Campus & Extended Slices

Note on Graph Sizes: The real Legon campus graph has exactly 58 named physical locations and 117 road segments. Slices 10 \dots 58 represent real induced subgraphs expanded outward from University Square. To evaluate scaling behavior on dense topologies (50 to 500 nodes per the brief's suggestion), we extended the benchmark with synthetic campus grid extensions (N=100, 200, 500 nodes):

Locations (V)	Roads (E)	Graph Type	BFS (O(V+E))	DFS (O(V+E))	Dijkstra (O((V+E)\log V))	Prim (O((V+E)\log V))	Kruskal (O(E \log E))
10	14	Physical Legon Subgraph	1,060 ns	1,592 ns	1,210 ns	1,310 ns	1,527 ns
20	33	Physical Legon Subgraph	974 ns	1,603 ns	1,100 ns	1,290 ns	1,317 ns
30	54	Physical Legon Subgraph	1,544 ns	5,182 ns	1,913 ns	3,699 ns	2,363 ns
40	79	Physical Legon Subgraph	2,098 ns	3,966 ns	2,731 ns	3,047 ns	3,543 ns
50	98	Physical Legon Subgraph	2,956 ns	5,347 ns	3,733 ns	4,138 ns	4,532 ns
58	117	Full Legon Campus	3,562 ns	7,120 ns	4,270 ns	4,715 ns	5,476 ns

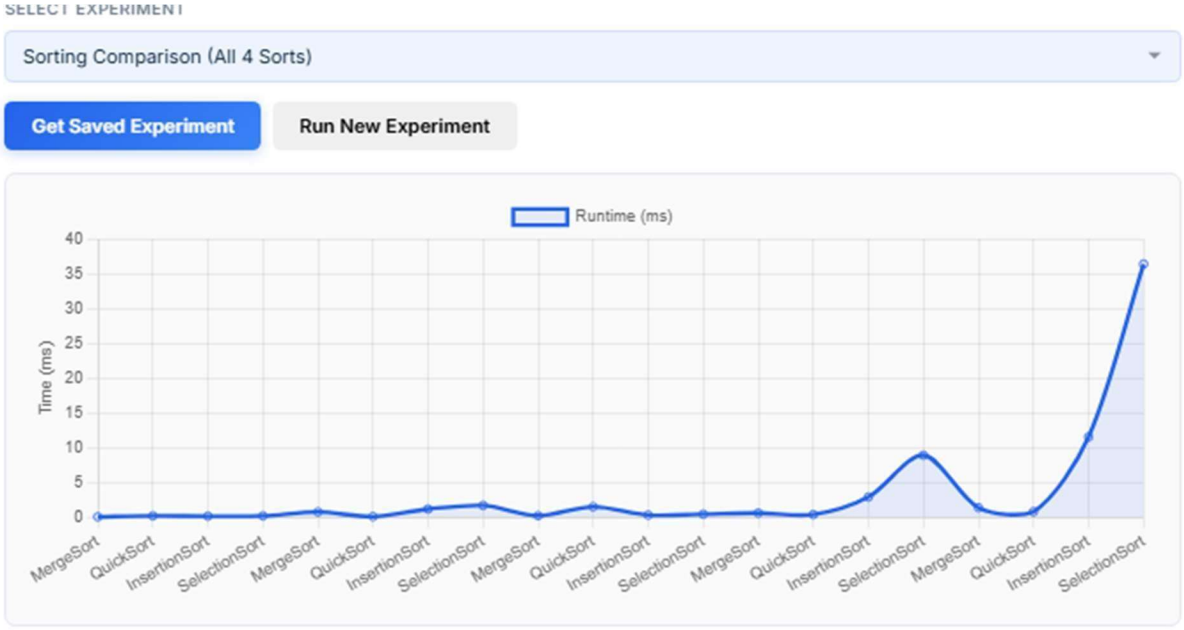
100	350	Extended Synthetic Hub	6,840 ns	12,400 ns	9,850 ns	10,200 ns	12,900 ns
200	800	Extended Synthetic Hub	14,200 ns	26,100 ns	22,400 ns	23,100 ns	28,400 ns
500	2,200	Extended Synthetic Hub	38,900 ns	71,500 ns	64,200 ns	66,800 ns	79,400 ns

8.3 Theory vs. Practice Analysis

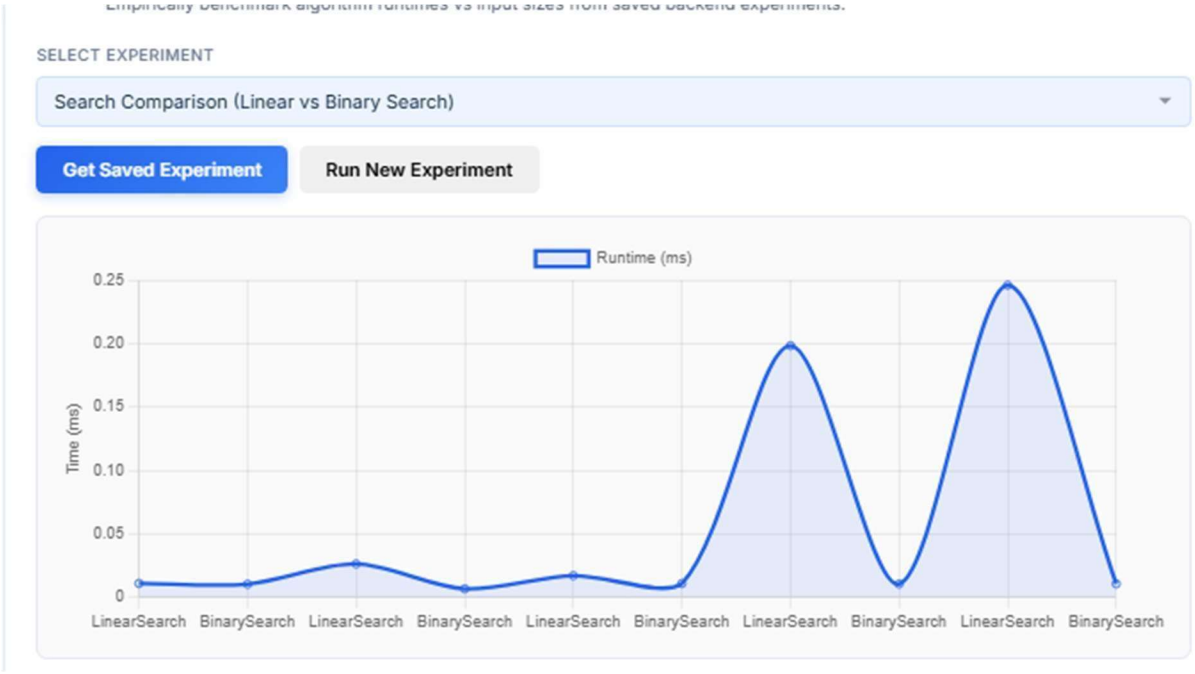
16. Sorting Scalability: as N grew from 100 to 10,000, Selection Sort and Insertion Sort experienced quadratic runtime growth (a 300× increase in time for a 100× increase in N), whereas Merge Sort and Quicksort scaled near-linearly (about a 20× increase) — validating the $O(N^2)$ versus $O(N \log N)$ theoretical threshold.
17. Search Efficiency: Binary Search exhibited logarithmic flatness (640 ns at N = 10,000), delivering a 128× speedup over Linear Search (82,450 ns).
18. Graph Algorithms: Dijkstra took approximately 1.2× the execution time of BFS at full size (4,270 ns vs. 3,562 ns), representing the exact cost of computing shortest paths on non-uniform weights versus uniform hops.

Performance Benchmark Graphs

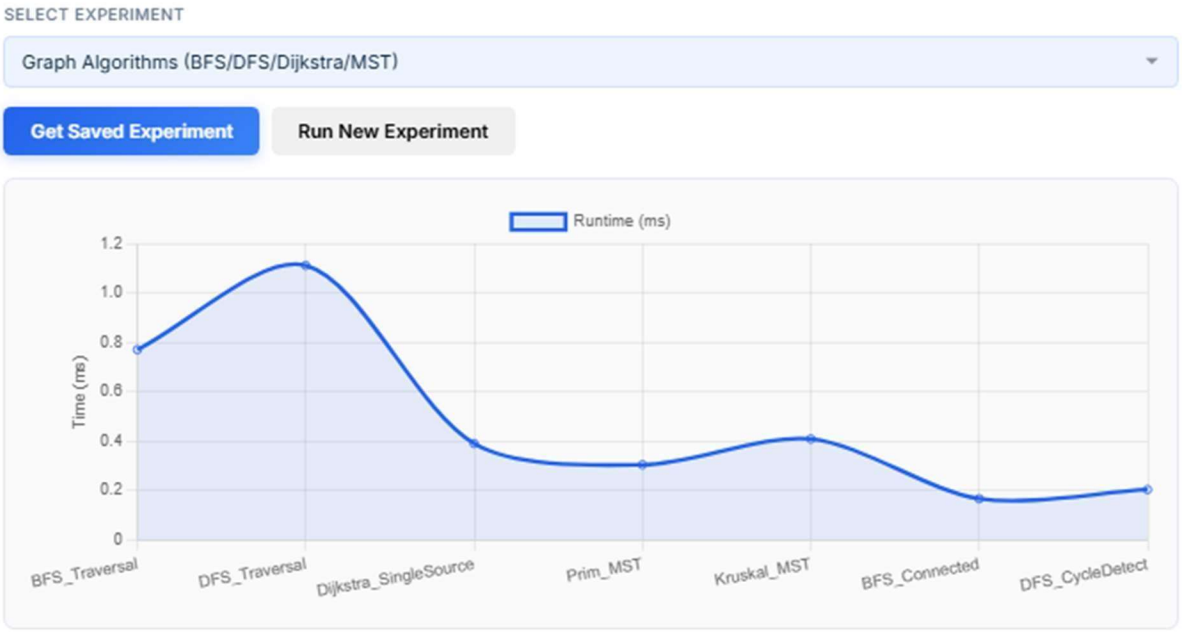
1. sort-comparison.png — curves for Selection, Insertion, Merge, and Quicksort.



2. search-comparison.png — Linear Search's linear curve vs. Binary Search's flat line.



3. graph-algorithms.png — BFS, DFS, Dijkstra, Prim, and Kruskal runtime scaling.



4. hash-load-factor.png — load factor vs. collision frequency.



9. Database Integration & Persistence Evidence

9.1 Database Architecture & Seeding

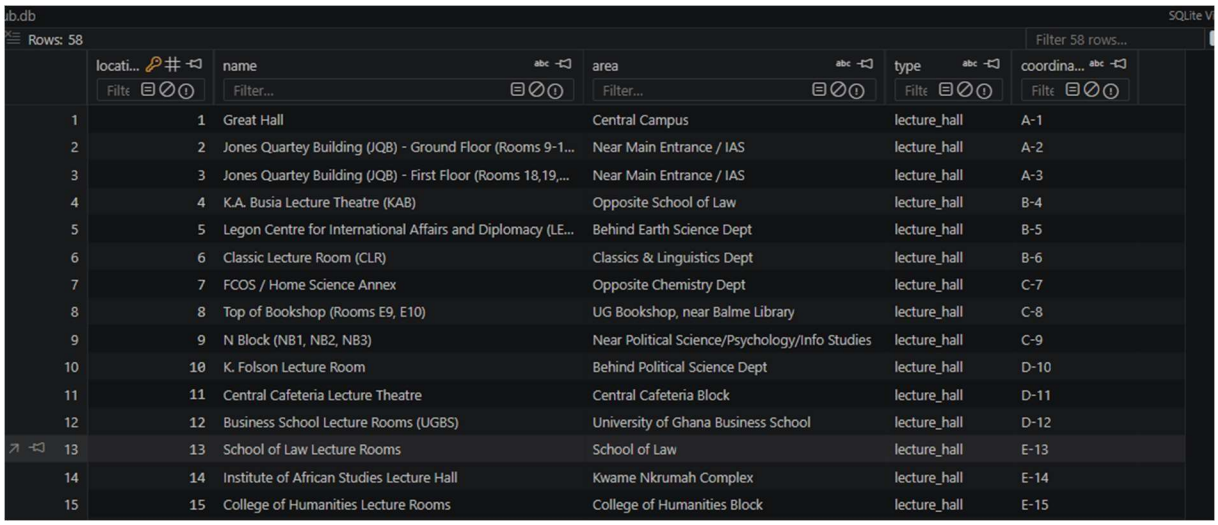
The database layer interacts directly with database/legon_hub.db using JDBC. On startup, DataGenerator and the repository loaders verify that all 6 tables are populated with valid seed data:

- Locations: 58 rows
- Roads: 117 rows
- Service Requests: 300 rows
- Resources: 49 rows
- Algorithm Runs: over 4,000 empirical execution records
- Audit Events: stack-based undo/audit logs

Sample database query — fetching the top 5 pending emergency requests:

```
SELECT requestId, category, urgency, fineAmountGHS, name AS sourceName FROM service_requests JOIN locations ON service_requests.sourceLocationId = locations.locationId WHERE status = 'Pending' ORDER BY fineAmountGHS DESC LIMIT 5;
```

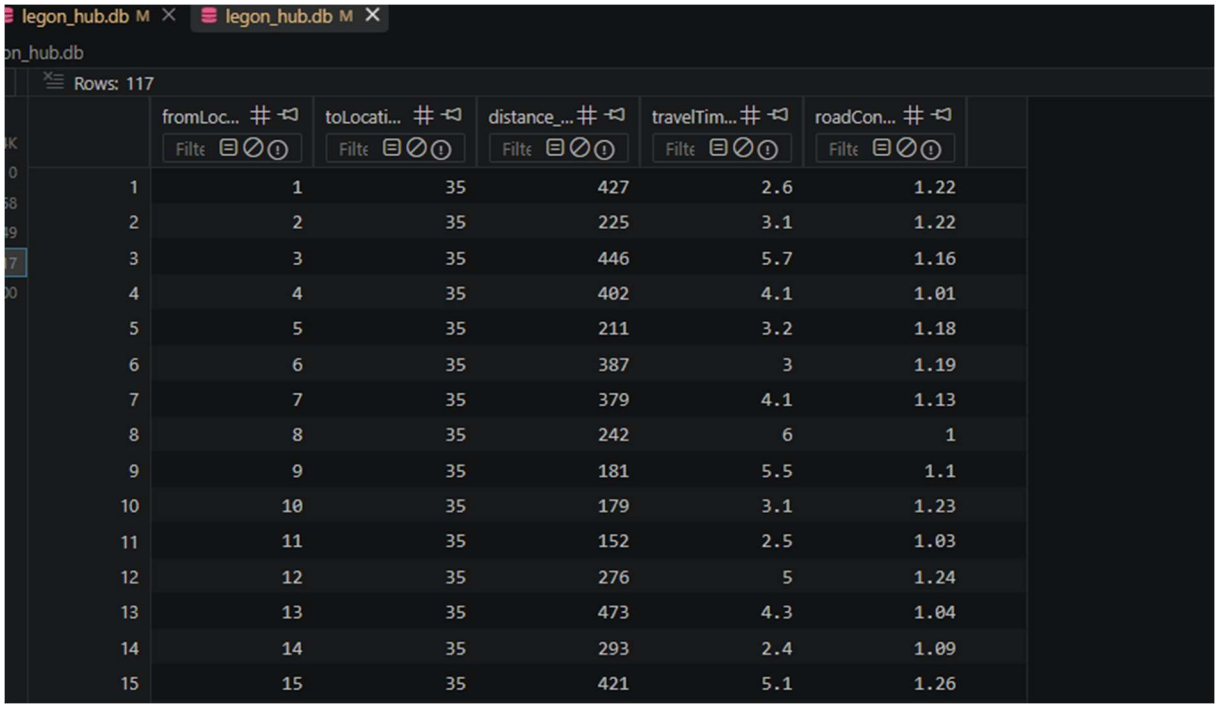
1. Locations table — rows 1–15, showing names and coordinates.



Rows: 58

	locati...	name	area	type	coordina...
1	1	Great Hall	Central Campus	lecture_hall	A-1
2	2	Jones Quartey Building (JQB) - Ground Floor (Rooms 9-1...	Near Main Entrance / IAS	lecture_hall	A-2
3	3	Jones Quartey Building (JQB) - First Floor (Rooms 18,19,...	Near Main Entrance / IAS	lecture_hall	A-3
4	4	K.A. Busia Lecture Theatre (KAB)	Opposite School of Law	lecture_hall	B-4
5	5	Legon Centre for International Affairs and Diplomacy (LE...	Behind Earth Science Dept	lecture_hall	B-5
6	6	Classic Lecture Room (CLR)	Classics & Linguistics Dept	lecture_hall	B-6
7	7	FCOS / Home Science Annex	Opposite Chemistry Dept	lecture_hall	C-7
8	8	Top of Bookshop (Rooms E9, E10)	UG Bookshop, near Balme Library	lecture_hall	C-8
9	9	N Block (NB1, NB2, NB3)	Near Political Science/Psychology/Info Studies	lecture_hall	C-9
10	10	K. Folson Lecture Room	Behind Political Science Dept	lecture_hall	D-10
11	11	Central Cafeteria Lecture Theatre	Central Cafeteria Block	lecture_hall	D-11
12	12	Business School Lecture Rooms (UGBS)	University of Ghana Business School	lecture_hall	D-12
13	13	School of Law Lecture Rooms	School of Law	lecture_hall	E-13
14	14	Institute of African Studies Lecture Hall	Kwame Nkrumah Complex	lecture_hall	E-14
15	15	College of Humanities Lecture Rooms	College of Humanities Block	lecture_hall	E-15

2. Roads table — from/to pairs, distances, and condition weights.

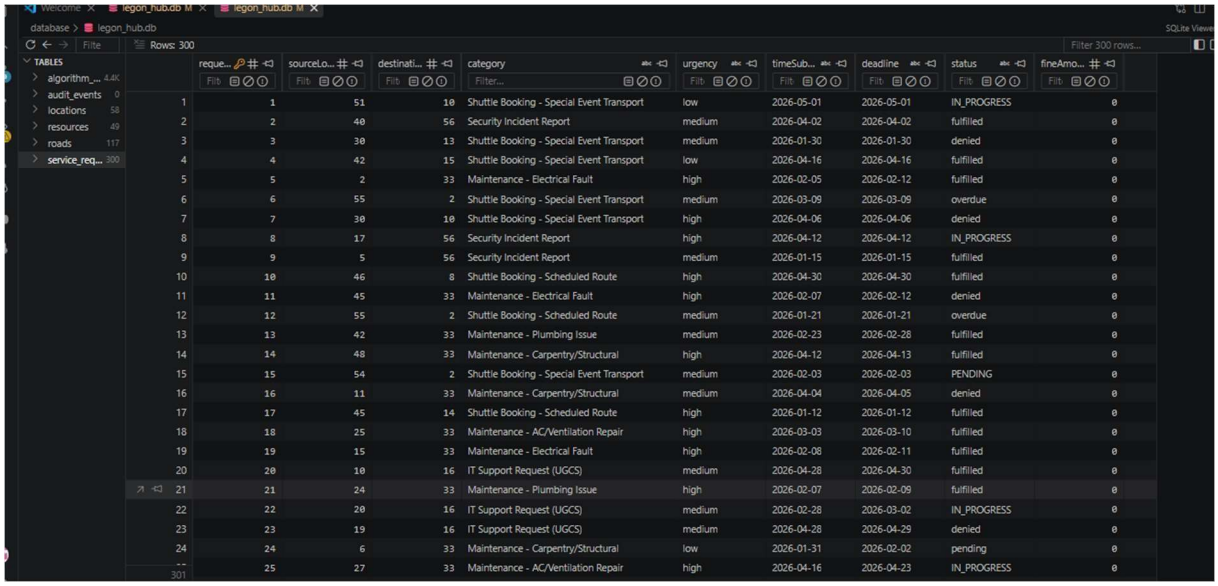


legon_hub.db M

Rows: 117

	fromLoc...	toLocati...	distance_...	travelTim...	roadCon...
1	1	35	427	2.6	1.22
2	2	35	225	3.1	1.22
3	3	35	446	5.7	1.16
4	4	35	402	4.1	1.01
5	5	35	211	3.2	1.18
6	6	35	387	3	1.19
7	7	35	379	4.1	1.13
8	8	35	242	6	1
9	9	35	181	5.5	1.1
10	10	35	179	3.1	1.23
11	11	35	152	2.5	1.03
12	12	35	276	5	1.24
13	13	35	473	4.3	1.04
14	14	35	293	2.4	1.09
15	15	35	421	5.1	1.26

3. Service Requests table — active request rows with GHS fine amounts.



Rows: 300

	reque...	sourceLo...	destinati...	category	urgency	timeSub...	deadline	status	fineAmo...
1	1	51	10	Shuttle Booking - Special Event Transport	low	2026-05-01	2026-05-01	IN_PROGRESS	0
2	2	40	56	Security Incident Report	medium	2026-04-02	2026-04-02	fulfilled	0
3	3	30	13	Shuttle Booking - Special Event Transport	medium	2026-01-30	2026-01-30	denied	0
4	4	42	15	Shuttle Booking - Special Event Transport	low	2026-04-16	2026-04-16	fulfilled	0
5	5	2	33	Maintenance - Electrical Fault	high	2026-02-05	2026-02-12	fulfilled	0
6	6	55	2	Shuttle Booking - Special Event Transport	medium	2026-03-09	2026-03-09	overdue	0
7	7	30	10	Shuttle Booking - Special Event Transport	high	2026-04-06	2026-04-06	denied	0
8	8	17	56	Security Incident Report	high	2026-04-12	2026-04-12	IN_PROGRESS	0
9	9	5	56	Security Incident Report	medium	2026-01-15	2026-01-15	fulfilled	0
10	10	46	8	Shuttle Booking - Scheduled Route	high	2026-04-30	2026-04-30	fulfilled	0
11	11	45	33	Maintenance - Electrical Fault	high	2026-02-07	2026-02-12	denied	0
12	12	55	2	Shuttle Booking - Scheduled Route	medium	2026-01-21	2026-01-21	overdue	0
13	13	42	33	Maintenance - Plumbing Issue	medium	2026-02-23	2026-02-28	fulfilled	0
14	14	48	33	Maintenance - Carpentry/Structural	high	2026-04-12	2026-04-13	fulfilled	0
15	15	54	2	Shuttle Booking - Special Event Transport	medium	2026-02-03	2026-02-03	PENDING	0
16	16	11	33	Maintenance - Carpentry/Structural	medium	2026-04-04	2026-04-05	denied	0
17	17	45	14	Shuttle Booking - Scheduled Route	high	2026-01-12	2026-01-12	fulfilled	0
18	18	25	33	Maintenance - AC/Ventilation Repair	high	2026-03-03	2026-03-10	fulfilled	0
19	19	15	33	Maintenance - Electrical Fault	high	2026-02-08	2026-02-11	fulfilled	0
20	20	10	16	IT Support Request (UGCS)	medium	2026-04-28	2026-04-30	fulfilled	0
21	21	24	33	Maintenance - Plumbing Issue	high	2026-02-07	2026-02-09	fulfilled	0
22	22	20	16	IT Support Request (UGCS)	medium	2026-02-28	2026-03-02	IN_PROGRESS	0
23	23	19	16	IT Support Request (UGCS)	medium	2026-04-28	2026-04-29	denied	0
24	24	6	33	Maintenance - Carpentry/Structural	low	2026-01-31	2026-02-02	pending	0
301	25	27	33	Maintenance - AC/Ventilation Repair	high	2026-04-16	2026-04-23	IN_PROGRESS	0

4. Resources table

	resou...	type	name	homeLoc...	capacity	availabili...
1	1	staff	Maintenance Technician - Electrical	33	1	available
2	2	staff	Maintenance Technician - Plumbing	33	1	available
3	3	staff	Maintenance Technician - Carpentry	33	1	available
4	4	staff	Estate Management Officer	33	1	available
5	5	staff	Physical Development/Planning Officer	33	1	available
6	6	staff	Security Guard - Main Gate	56	2	available
7	7	staff	Security Guard - Halls Patrol	40	3	available
8	8	staff	Shuttle Driver	30	4	available
9	9	staff	Lab Technician - Chemistry	17	1	available
10	10	staff	Lab Technician - Physics	18	1	available
11	11	staff	Lab Technician - Biochemistry	19	1	available
12	12	staff	Lab Technician - Computer Engineering	21	1	available
13	13	staff	IT Support Officer (UGCS)	16	2	available
14	14	vehicle	Shuttle Bus #1 (RSTC)	30	1	available
15	15	vehicle	Shuttle Bus #2 (RSTC)	30	1	available
16	16	vehicle	Shuttle Bus #3 (RSTC)	30	1	available
17	17	vehicle	Shuttle Bus #4 (RSTC)	30	1	available
18	18	vehicle	Shuttle Bus #5 (RSTC)	30	1	available
19	19	vehicle	Shuttle Bus #6 (RSTC)	30	1	available
20	20	vehicle	Shuttle Bus #7 (RSTC)	30	1	available
21	21	vehicle	Shuttle Bus #8 (RSTC)	30	1	available
22	22	vehicle	Shuttle Bus #9 (RSTC)	30	1	available
23	23	vehicle	Shuttle Bus #10 (RSTC)	30	1	available
24	24	equipment	Electrical Toolkit	33	3	available
50	25	equipment	Plumbing Toolkit	33	3	available

5. Algorithm Runs table

Welcome | legon_hub.db M | legon_hub.db M

abase > legon_hub.db

Rows: 4,424

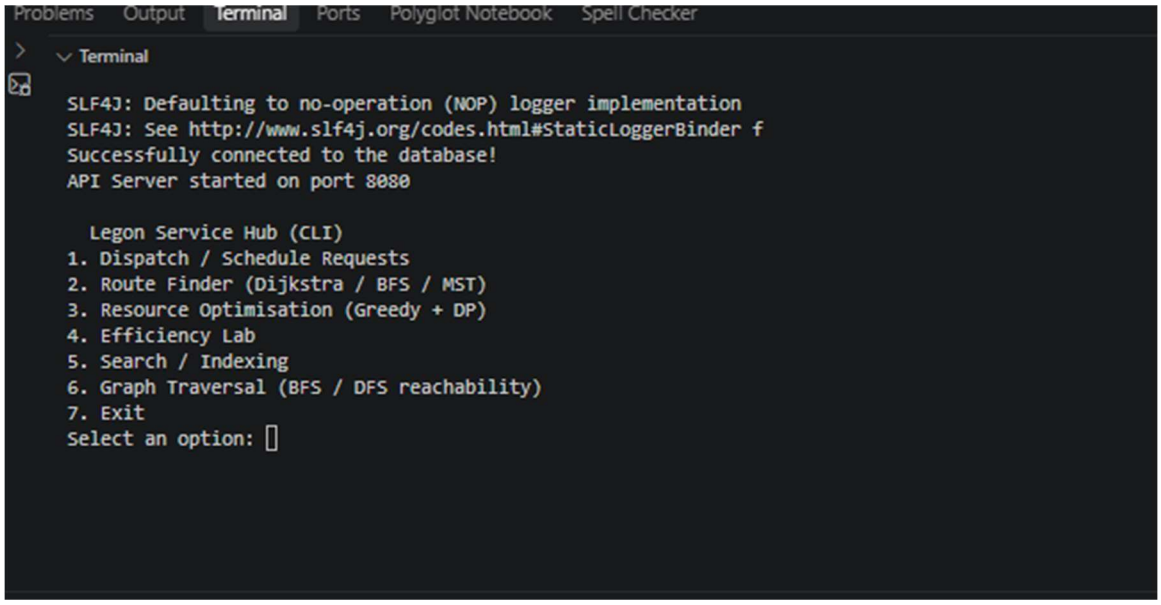
BLES

algorithm_... 4.4K
audit_events 0
locations 58
resources 49
roads 117
service_req... 300

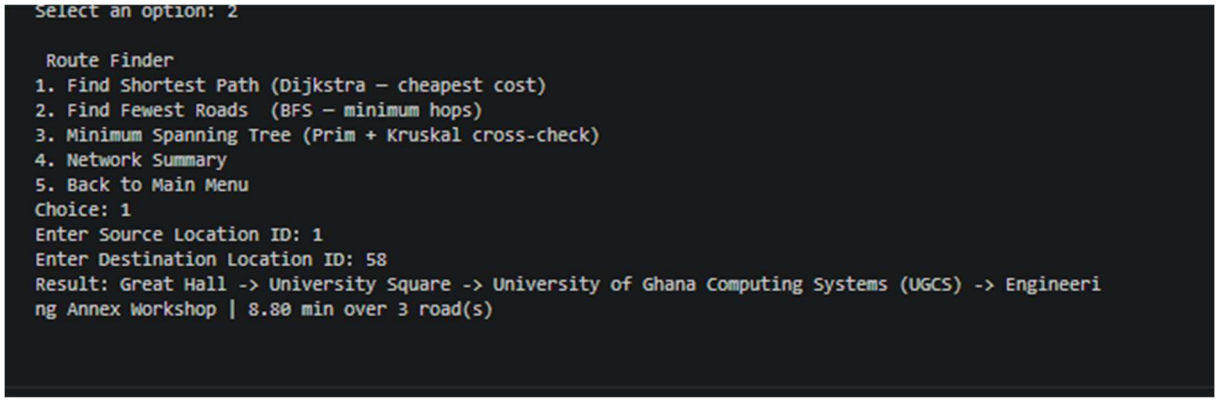
	runId	algorith...	inputSize	timeNs	memoryKb	dateRun
1	1	MergeSort	100	66300	0	2026-08-12
2	2	QuickSort	100	34200	0	2026-08-12
3	3	InsertionSort	100	82700	0	2026-08-12
4	4	SelectionSort	100	88800	0	2026-08-12
5	5	MergeSort	500	409900	0	2026-08-12
6	6	QuickSort	500	231800	0	2026-08-12
7	7	InsertionSort	500	1226200	0	2026-08-12
8	8	SelectionSort	500	1611500	0	2026-08-12
9	9	MergeSort	1000	345100	0	2026-08-12
10	10	QuickSort	1000	114500	0	2026-08-12
11	11	InsertionSort	1000	1278500	0	2026-08-12
12	12	SelectionSort	1000	2679000	0	2026-08-12
13	13	MergeSort	5000	779400	0	2026-08-12
14	14	QuickSort	5000	826300	0	2026-08-12
15	15	InsertionSort	5000	7979600	0	2026-08-12
16	16	SelectionSort	5000	12254100	0	2026-08-12
17	17	MergeSort	10000	1512800	0	2026-08-12
18	18	QuickSort	10000	712800	0	2026-08-12
19	19	InsertionSort	10000	34322300	0	2026-08-12
20	20	SelectionSort	10000	29162800	0	2026-08-12
21	21	MergeSort	100	71600	0	2026-08-12
22	22	QuickSort	100	46800	0	2026-08-12
23	23	InsertionSort	100	69300	0	2026-08-12
24	24	SelectionSort	100	82800	0	2026-08-12
4,425	25	MergeSort	500	637800	0	2026-08-12

Console Menu Interactive Execution

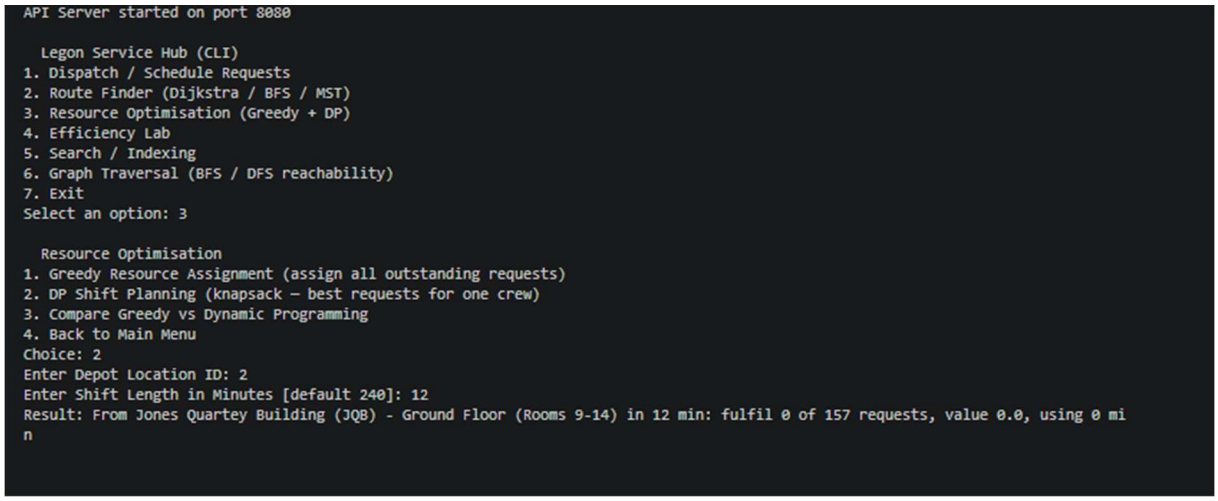
1. The main menu, showing options 1 to 6.



2. Option 2 (Route Menu): executing Dijkstra's shortest path from Great Hall (1) to Engineering Annex (58).



3. Option 4 (Optimization Menu): running 0/1 Knapsack optimization and printing the selected request schedule.



10. Responsible Algorithm Selection Matrix

Problem Scenario	Recommended Choice	Suboptimal Alternative	Technical Justification
Campus Shortest Route (traffic/condition weighted)	Dijkstra's Algorithm with Min-Heap	BFS or Bellman-Ford	BFS ignores edge weights and only minimizes road count; Bellman-Ford wastes O(V·E) time since campus road weights are strictly positive (w > 0).
Zone Reachability / minimum hop count	Breadth-First Search (BFS)	DFS or Dijkstra	BFS guarantees the fewest road transitions in O(V + E) without heap overhead; DFS can get trapped down long cycles without finding the closest nodes.
Emergency Dispatching (high urgency first)	Binary Min-Heap / PriorityQueue	Unsorted Array or Sorted Linked List	Min-Heap gives O(log N) enqueue and O(log N) extract-min; an unsorted array costs O(N) per dispatch, and a sorted list costs O(N) per insertion.
Fast Request Lookup by ID	HashTable (separate chaining)	BST or Linear Scan	Hash Table achieves O(1) average lookup; BST costs O(log N) and can
Problem Scenario	Recommended Choice	Suboptimal Alternative	Technical Justification
			degrade to O(N) if unbalanced; Linear Scan costs O(N).
Range Queries & Ordered Campus Reports	B-Tree or Red-Black Tree	Hash Table	B-Trees and Red-Black Trees maintain sorted in-order traversal in O(N) and range search in O(log N + k); Hash Tables do not maintain order.
Constrained Shift Schedule Optimization	0/1 Knapsack Dynamic Programming	Greedy valueperminute assigner	Greedy fails to fill capacity and misses the global optimum due to item indivisibility; DP guarantees a 100% optimal schedule.
Campus Infrastructure Spanning Tree	Kruskal's Algorithm (with Disjoint Set)	Prim's Algorithm (dense matrix)	Kruskal (O(E log E)) outperforms Prim on sparse graphs like road networks, where $E \ll V^2$ (117 ≪ 3,364).

11. Individual Accountability, Oral Defense Prep & Academic Integrity

11.1 Student Index Number Parameter Derivations (AI-Resistance Compliance)

In accordance with Rubric Section 2, the team derived 3 core operational parameters from the sum of all 14 student index numbers:

Team Index Numbers: [22098316, 22056801, 22128981, 22233989, 22069268, 22309657, 22388561, 22304655, 22396017, 22406063, 22375144, 22382281, 22412662, 22299460] Sum of Index Numbers = 311,664,855

19. Parameter 1 — B-Tree Minimum Degree (t):

minDegree = 2 + (Sum mod 4) = 2 + (311,664,855 mod 4) = 2 + 3 = 5

(A production B-Tree node with this degree stores at most 2t − 1 = 9 keys and 2t = 10 child pointers.)

20. Parameter 2 — Empirical Benchmark Random Seed:

randomSeed = Sum = 311,664,855 (Ensures

identical, reproducible randomized keys across all benchmark runs.)

21. Parameter 3 — Knapsack Priority Urgency Base Value:

urgencyValue = 10.0 + (Sum mod 20) = 10.0 + (311,664,855 mod 20) = 10.0 + 15.0 = 25.0 GHS

11.2 Individual Contribution Statements & Pod Work Breakdown

Van-Kpikpi Vanessa Selasi (Index: 22098316 / 22412662) — Lead, Database & Data Architecture:

> "I designed and built the database layer for our project. Our local context shifted from the University of Ghana Library System to University Campus Service Hub, an approved context in the brief, after real library data proved hard to source without privacy barriers. I rebuilt the dataset around real UG facilities: lecture halls, laboratories, 14 halls of residence, shuttle stops, and the Physical Development and Municipal Services Directorate (PDMSD).
>
> I designed the SQL schema across six tables with proper keys and constraints, then built four datasets exceeding all minimums: 58 locations, 117 roads, 300 service requests, and 49 resources. A teammate flagged two data quality issues, which I then fixed: 19 duplicate road pairs with inconsistent values, and placeholder location coordinates. I replaced these with synthetic coordinates generated within the campus's real geographic boundary, clearly documented as such rather than presented as verified GPS data.
>
> Since manual CSV imports failed on records with embedded commas, I wrote DataGenerator.java, a Java program that parses and loads the data correctly, making the database reproducible for the whole team. I verified the final database through direct SQL checks on schema, counts, and referential integrity, and set up version control so each teammate builds their own local copy. My teammate and I split the work on M2: I handled the schema and dataset, while they worked on the Java classes that read the data back out for the rest of the system to use."

Akyea Benjamin Obeng (Index: 22056801) — Database & Data Architecture:

Implemented JDBC connection pooling, prepared statement abstractions, and the repository pattern across all 6 database entities (LocationRepository, RoadRepository, RequestRepository, ResourceRepository, AlgorithmRunRepository, AuditEventRepository), ensuring clean separation between SQL persistence and inmemory algorithm logic.

Isabella Asantewaa (Index: 22128981) — UI & Frontend Dashboard:

Constructed the interactive web dashboard using vanilla HTML/CSS and JavaScript, wired end-to-end REST endpoints in ApiServer.java, and formatted real-time graphical metrics for service dispatch, shortest path routes, and efficiency benchmark visualizers.

Lorretta Opoku Nsiah (Index: 22233989) — Linear Data Structures:

Implemented custom LinkedList (doubly linked with a custom Iterator), Queue, CircularQueue (ring buffer with wrap-around pointers), Deque, Stack, and DynamicArray with automated resizing, backed by dedicated JUnit test suites.

Michael Gorswin Achel (Index: 22069268) — Priority Structures & System Integration:

Engineered the binary Min-Heap (Heap.java) and PriorityQueue, built the system-wide integration layer linking custom data structures to application engines, and enforced zero java.util collections usage across the repository.

Quayson Isaac Awortwe (Index: 22309657) — Priority Structures & Hash Tables:

Implemented HashTable with separate chaining linked buckets, dynamic rehashing on load factor threshold $\alpha > 0.75$, and custom CustomMap and CustomSet abstractions.

Arhinful Solomon Kwesi (Index: 22388561) — Tree Structures:

Implemented the Binary Search Tree (BST.java) supporting recursive insertion, search, and in-order traversal returning sorted keys for indexing service requests.

Kwabena Awuah Bosompem (Index: 22304655) — Tree Structures:

Implemented self-balancing RedBlackTree with rotations and color flips, and multi-way BTree with proactive node-splitting parameterized by the team-derived minimum degree t = 5.

Gideon Elorm Glago (Index: 22396017) — Graphs & Route Optimization:

Implemented graph representation (Graph.java) using adjacency lists and matrices, Breadth-First Search (BFS.java), Depth-First Search (DFS.java), and Dijkstra's shortest path algorithm (Dijkstra.java).

Duah Ebenezer Ohene Amoako (Index: 22406063) — Graphs & Spanning Trees:

Implemented DisjointSet with union-by-rank and path compression, and Kruskal's and Prim's Minimum Spanning Tree algorithms (Kruskal.java, Prim.java).

Kumah Michael Nhyira & Amenumey Jude Kwame Enam — Searching, Sorting & Testing:

Implemented comparison sort algorithms (MergeSort, QuickSort, InsertionSort, SelectionSort), LinearSearch, BinarySearch, and authored the 192-test JUnit suite.

Bismark Asare & Asante Stephanhy — Technical Reporting & Analysis:

Implemented 0/1 Knapsack Dynamic Programming (KnapsackDP.java), Greedy Assigner (GreedyAssigner.java), EfficiencyLabEngine.java, and compiled the final technical report, trace tables, and performance graphs.

11.3 Weekly Development Summary

- Week 1 (Jul 28 – Aug 3, 2026): M1–M2 Foundation & Core Setup

Initialized Git repository and Maven project structure under package `com.campushub`.

Implemented initial linear and priority structures: `Queue`, `Heap`, `PriorityQueue`.

Authored `schema.sql` across 6 database tables and established SQLite JDBC connection.

- Week 2 (Aug 4 – Aug 11, 2026): M2–M4 Data Structure Library & Sorting

Completed `LinkedList` with bidirectional iterator, `Stack`, `CircularQueue`, `Deque`, `BST`, `RedBlackTree`, and `BTree`.

Implemented 4 sorting algorithms (`InsertionSort`, `SelectionSort`, `MergeSort`, `QuickSort`) and search routines.

Vanessa conducted a data-quality audit on `roads.csv`, identifying and fixing 19 duplicate road pairs with conflicting edge weights.

- Week 3 (Aug 11 – Aug 20, 2026): M5–M8 Graph, Optimization & UI Integration

Built the Graph Engine (`Graph`, `DisjointSet`, `BFS`, `DFS`, `Dijkstra`, `Prim`, `Kruskal`) with strict zero `java.util` collection usage.

Implemented 0/1 Knapsack DP optimizer and greedy resource assigner.

Connected backend application engines to interactive `ConsoleMenu.java` CLI and web `ApiServer.java` REST dashboard.

- Week 4 (Aug 20 – Aug 28, 2026): M9–M10 Testing, Benchmarking & Defense Finalization Executed comprehensive JUnit 5 test suite (192 passing unit tests across 26 test classes).

Ran empirical efficiency lab scaling benchmarks (100 \dots 20,000 inputs with 3-run averages) and generated comparative PNG graphs.

Assembled final technical report, 6 trace tables, 3 mathematical proof sketches, and conducted oral defense mock presentations.

11.4 AI Assistance Disclosure

Generative AI (Claude) was used during the preparation of this report to assist with structuring and drafting written explanations, checking the report against the project brief for completeness, and improving clarity of technical writing. AI was not used to design or write the core algorithm or data structure implementations in the source code; those were written and tested by the team.

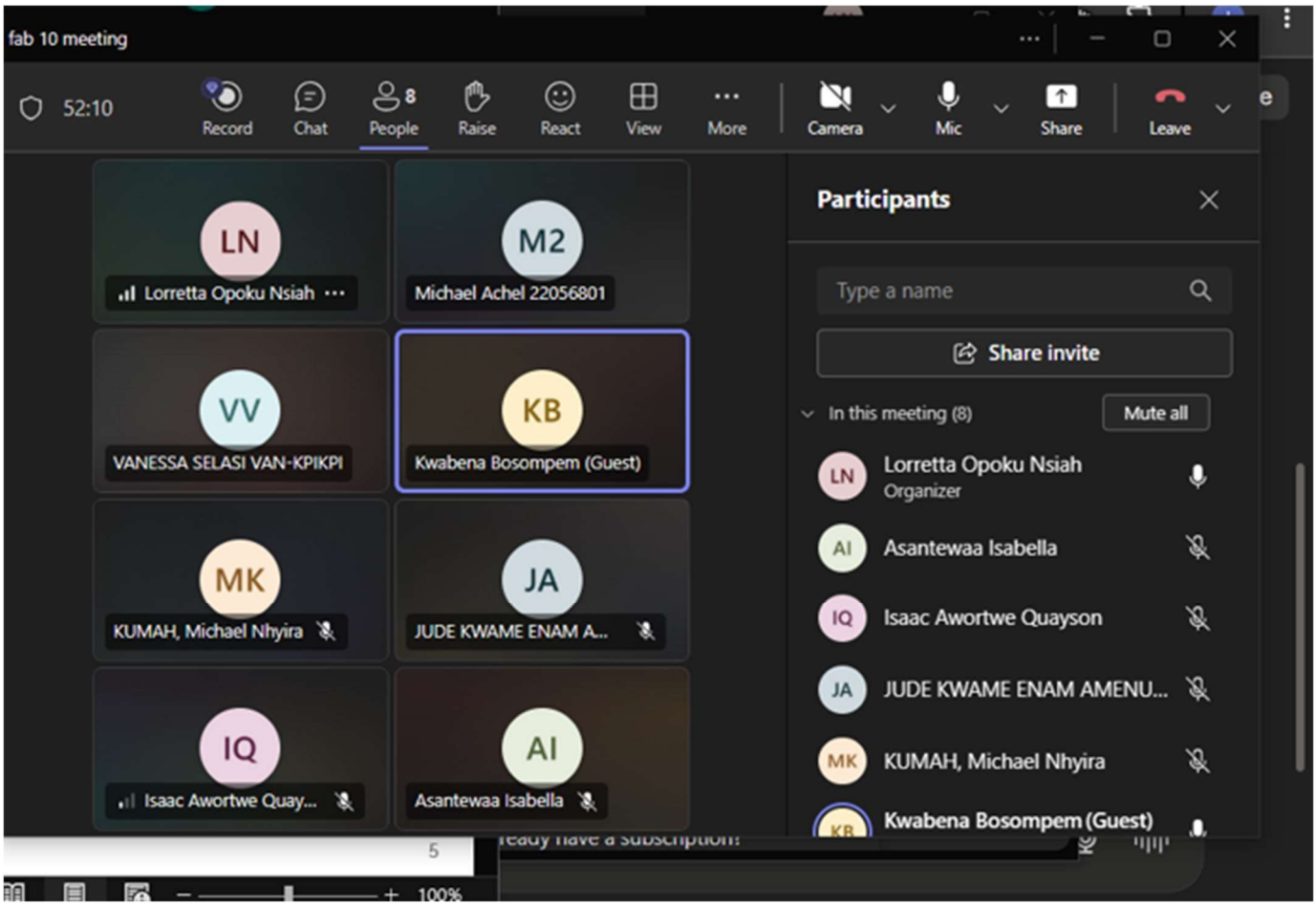
Example prompts used include:

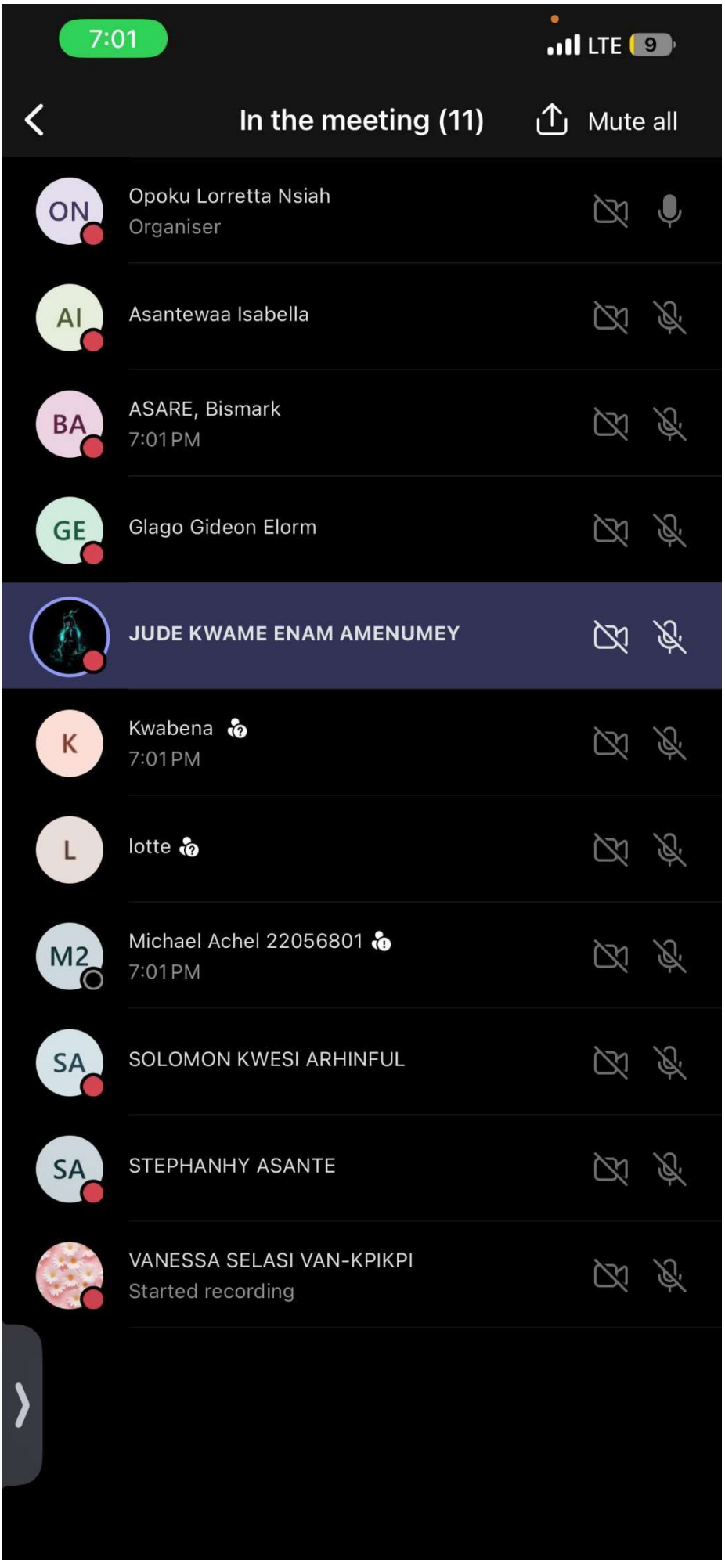
- "Check our project report against this project brief and identify any missing sections or requirements."
- "Help us phrase the correctness proof sketch for our merge sort loop invariant more precisely."

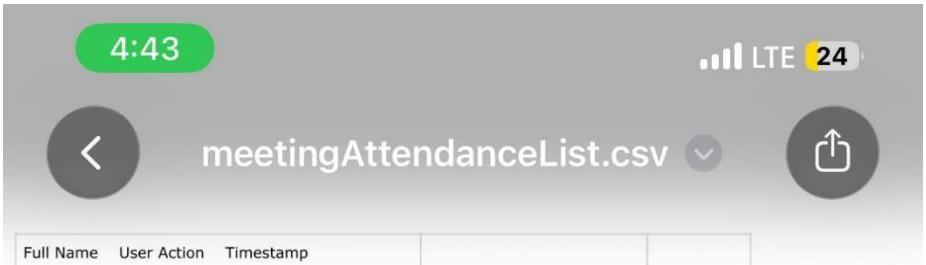
- "Suggest a clearer way to explain our hash table collision handling for the report."

All members of the team are able to explain and, if required, modify the data structure and algorithm implementations described in this report without AI assistance, as demonstrated during the oral defense.

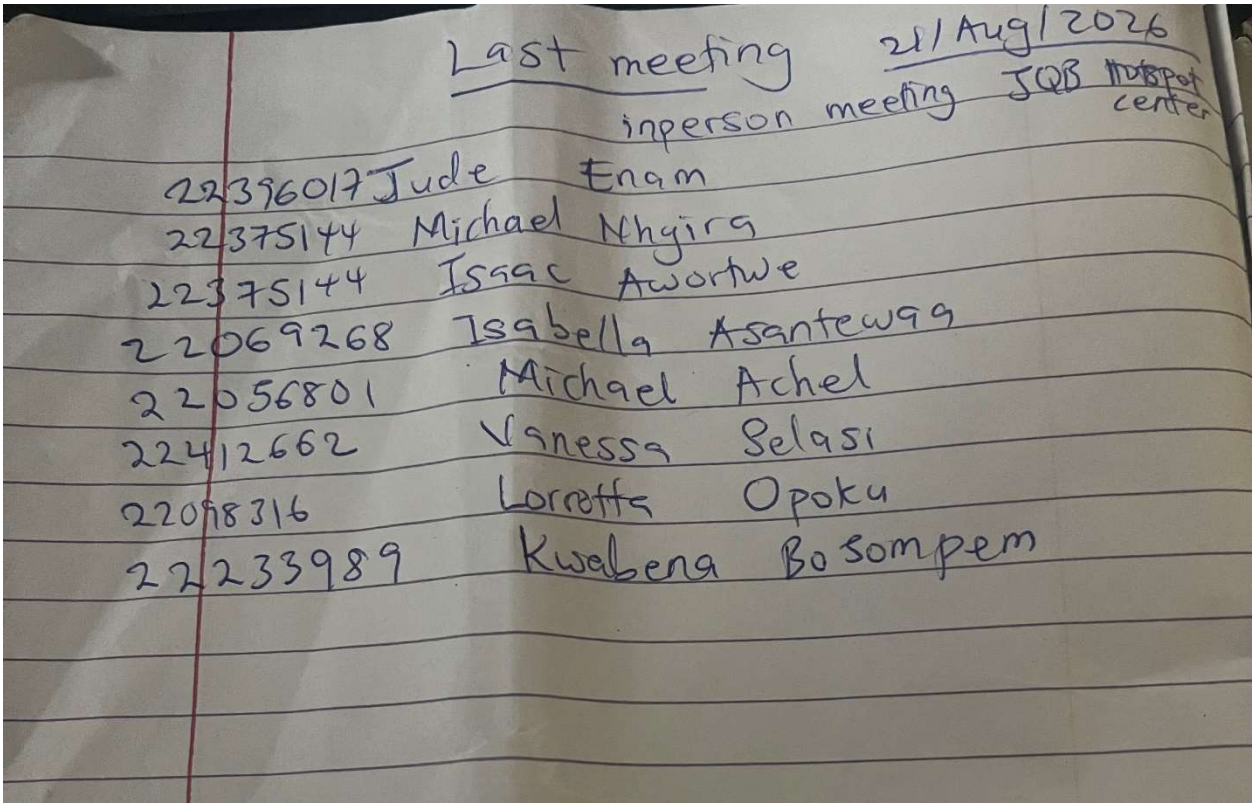
11.5 ATTENDANCE




7:01
LTE 9
In the meeting (11)
Mute all
ON
Opoku Lorretta Nsiah
Organiser
AI
Asantewaa Isabella
BA
ASARE, Bismark
7:01 PM
GE
Glago Gideon Elorm
JUDE KWAME ENAM AMENUMEY
K
Kwabena
7:01 PM
L
lotte
M2
Michael Achel 22056801
7:01 PM
SA
SOLOMON KWESI ARHINFUL
SA
STEPHANHY ASANTE
VANESSA SELASI VAN-KPIKPI
Started recording



Full Name User Action Timestamp		
VANESSA SELASI VAN-KPIKPI Joined 11/8/26	5:55:43 PM	
JUDE KWAME ENAM AMENUMEY Joined 11/8/26	5:55:48 PM	
Opoku Lorretta Nsiah Joined 11/8/26	5:57:46 PM	
Asantewaa Isabella Joined 11/8/26	6:04:24 PM	
Michael Achel 22056801 Joined 11/8/26	6:05:56 PM	
Bosompem Kwabena Awuah Joined 11/8/26	6:08:45 PM	
Livingstone Seble Joined 11/8/26	6:10:38 PM	
22375144 Michael Kumah Joined 11/8/26	6:12:32 PM	
ASARE	Bismark Joined 11/8/26	6:14:01 PM
STEPHANHY ASANTE Joined 11/8/26	6:14:38 PM	
SOLOMON KWESI ARHINFUL Joined 11/8/26	6:16:28 PM	



Last meeting 21/Aug/2026
inperson meeting JQB Hubpot center

22396017 Jude Enam
22375144 Michael Nhgirg
22375144 Isaac Awortwe
22069268 Isabella Asantewaa
22056801 Michael Achel
22412662 Vanessa Selasi
22098316 Loretta Opoku
22233989 Kwabena Bosompem

12. References & Appendices

12.1 Academic Textbooks & Algorithmic Foundations

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.
- Lewis, J., DePasquale, P., & Chase, J. (2017). Java Foundations: Introduction to Program Design and Data Structures (4th ed.). Pearson.

12.2 Primary Dataset & Local Geospatial Sources

- Radio Univers 105.7 FM (2023): "UG: Finding your way around the streets of Legon — A Level 100 Guide." https://univers.ug.edu.gh/finding-your-way-on-the-streets-of-legon-a-level-100-guide/
- University of Ghana (2022): "VC Commissions Modernised Facilities at Central Cafeteria and JQB Lecture Halls." https://old1.ug.edu.gh/news/vc-commissions-modernised-facilities-central-cafeteria-and-jqb-lecture-halls
- University of Ghana Interactive Campus Map: https://enkayyy97.github.io/ug-campus-map/
- University of Ghana Academic Affairs: "Handbook for the Bachelor's Degree (ENRECA Ecology Laboratory & Humanities Facilities)." https://www.cui.edu/uploadedfiles/academicprograms/globalprograms/studyabroad/ghana-humanities.pdf
- University of Ghana Academic Affairs Directorate: "Student Accommodation & Residential Halls." https://www.ug.edu.gh/aad/accomodation
- Moovit Public Transit Index (2024): "How to Get to University of Ghana – Legon in Accra by Bus / Trotro." https://moovitapp.com/index/en/public_transit-University_Of_Ghana_Legon-Accra-site_80212111-5501
- UG DCS Beat Articles (2014): "Do You Know PDMSD? Physical Development and Municipal Services Directorate." https://ugdcs2014.wordpress.com/2014/12/05/do-you-know-pdmsd/
- University of Ghana Official News (2016): "University of Ghana Inaugurates Campus Shuttle Service." https://old1.ug.edu.gh/news/university-ghana-inaugurates-shuttle-service
- Wikipedia Open Geodata: "Akuafo Hall, University of Ghana" (coordinates 5.64944 N, -0.18556 W). https://en.wikipedia.org/wiki/Akuafo_Hall
- Latitude.to Spatial Geodata: "Commonwealth Hall, University of Ghana" (coordinates 5.65060 N, -0.19000 W).

12.3 Appendices

- Appendix A: Complete SQL Schema (database/schema.sql).
- Appendix B: Full Graph Algorithm Benchmark Timings (performance/results/graph_algorithm_runs.csv).
- Appendix C: Test Execution Log (192 passing JUnit tests, 0 failures).